

Università degli Studi di Napoli
Federico II

A.A. 2026-2027

Cheat sheet
UML e design pattern

Indice

I I diagrammi UML 4

1 Il diagramma delle classi: classi, attributi, operazioni 5
2 Il diagramma delle classi: le relazioni 7
3 Dal codice al diagramma: attributi e operazioni 9
4 Dal codice al diagramma: riferimenti fra classi 12
5 Dal codice al diagramma: array e collezioni 15
6 Dal codice al diagramma: ereditarietà, interfacce, tipi speciali 19
7 Dal codice al diagramma: dipendenze e tutto-parti 22
8 Il diagramma dei casi d'uso 25
9 Il diagramma di sequenza 27
10 Package, componenti e deployment 29
11 Il diagramma di stato 31
12 Il diagramma di attività 33
13 Quale diagramma? 35

II Riconoscere i design pattern 36

14 Come si riconosce un pattern 37
15 Singleton 39
16 Builder 41
17 Adapter 44
18 Composite 46
19 Decorator 48
20 Observer 50
21 State 52
22 Strategy 54
23 Dependency Injection 56
24 Pattern a confronto 58

Come è fatto questo cheat sheet

Il cheat sheet raccoglie in poche pagine la notazione UML e i design pattern del corso, e accompagna la dispensa (capitoli 6, 7, 8, 9 e 15). Ha due parti.

- **Parte I, i diagrammi UML.** Per ogni diagramma le **voci** della notazione, una per riga: il numero e il nome, il disegno, il significato e, quando c'è, l'equivalente in Java. Per il diagramma delle classi ci sono poi i **casi «dal codice al diagramma»**: a sinistra un pezzo di codice Java, a destra il diagramma che gli corrisponde e la regola per passare dall'uno all'altro. I casi coprono tutto ciò che si può trovare in una classe Java: campi semplici e costanti, riferimenti ad altre classi, **array**, **liste**, insiemi, mappe e matrici, ereditarietà e interfacce, enumerazioni, classi generiche e annidate, dipendenze, tutto-parti. Gli ultimi argomenti traducono in codice anche i diagrammi di sequenza, di stato e di attività, e una tabella aiuta a scegliere il diagramma giusto.

- **Parte II, riconoscere i design pattern.** Si apre con un metodo per riconoscere un pattern dal testo di un esercizio, dal codice o dal diagramma, e con la tabella delle «firme» che distinguono i pattern che si somigliano. Poi una sezione per ogni pattern: la **carta d'identità** (intento, indizi nel testo, nel codice e nel diagramma, varianti, con che cosa non confonderlo), la **struttura** generale e un **esempio** completo, con il codice e il suo diagramma delle classi.

Il numero di ogni voce è un **collegamento**: nel PDF i rimandi come **1.6** portano alla voce citata; per chi stampa, l'indice dà le pagine. I colori distinguono i tipi di riquadro: **verde petrolio** per le voci di notazione, **blu** per i casi dal codice al diagramma, **viola** per i pattern, **arancione** per gli errori da evitare. L'etichetta oltre le slide segna ciò che nelle slide non c'è: costrutti di Java e notazioni UML che il corso non mostra (mappe, matrici, classi generiche, `record`, …), aggiunti per non trovarsi impreparati se un esercizio li contiene.

Tutto il codice è stato compilato con Java 21, e gli esempi dei pattern sono stati eseguiti: gli output riportati sono quelli reali. Per brevità negli esempi gli `import` di `java.util` sono sottintesi, e più classi stanno nello stesso file.

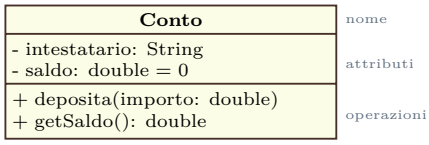
Parte I

I diagrammi UML

1 Il diagramma delle classi: classi, attributi, operazioni

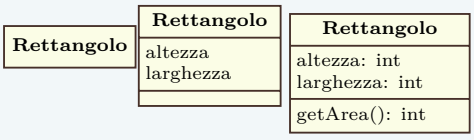
Dispensa: capitolo 7

1.1 Classe
rettangolo con tre scomparti



Il **nome** è in grassetto, al centro, al **singolare** (`Studente`, non `Studenti`). Sotto, gli **attributi** e le **operazioni**. In Java è una `class`: gli attributi sono i campi, le operazioni i metodi.

1.2 Livelli di dettaglio
regola delle informazioni soppresse



Tutti e tre sono la **stessa** classe. Ciò che manca non è assente: è solo **non mostrato** in quel diagramma. Nel modello di dominio bastano nomi e attributi; nel progetto di dettaglio si aggiungono tipi, visibilità e operazioni.

1.3 Sintassi di un attributo
solo il nome è obbligatorio

```
- nome [1]: String
= "Senza titolo" {readOnly}

visibilità nome molteplicità:
tipo = default {proprietà}
```

Un attributo **privato** di nome `nome`, con un solo valore, di tipo `String`, che vale all'inizio `"Senza titolo"` e non si può modificare. In Java: `private final String nome = "Senza titolo";`. Il tipo è un tipo primitivo o di libreria; se è una classe del diagramma, si disegna un'**associazione** (3.9).

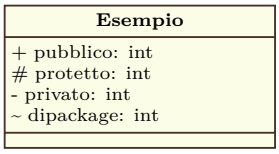
1.4 Sintassi di un'operazione
direzione dei parametri: in, out, inout

```
+ preleva(in importo: double):
        boolean {query}

visibilità nome(direzione nome:
    tipo = default, …): tipo
        {proprietà}
```

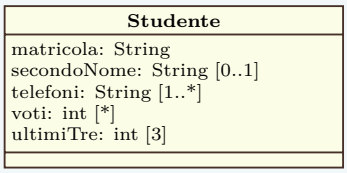
I parametri si scrivono `nome: tipo`, al contrario di Java. La direzione predefinita è `in` (in Java i parametri sono sempre `in`). Il tipo ritornato va dopo i due punti; `void` si omette. `{query}` dice che l'operazione non modifica l'oggetto oltre le slide.

1.5 Visibilità
per attributi e operazioni



`+`	tutte le classi	`public`
`#`	le sottoclassi	`protected`
`-`	solo la classe	`private`
`~`	lo stesso package	(nessun modificatore)

1.6 Molteplicità
quanti valori: array e collezioni



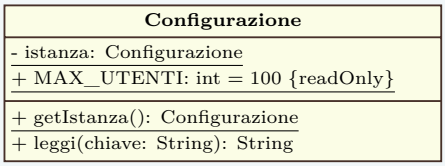
`1`	uno e uno solo (**predefinita**)
`0..1`	al più uno (può mancare: `null`)
`*`	quanti si vuole, anche nessuno (`0..*`)
`1..*`	almeno uno
`3 o 2..5`	esattamente 3; da 2 a 5

1.7 Ordinati e duplicati
{ordered}, {nonunique}: quale collezione Java

	senza duplicati	con duplicati
non ordinati	*(nessuna proprietà)* `Set`, `HashSet`	`{nonunique}` *nessuna classe Java*
ordinati	`{ordered}` `TreeSet`, `LinkedHashSet`	`{ordered, nonunique}` `List`, `ArrayList`, array

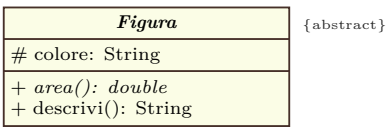
Per UML gli elementi di una molteplicità sono un **insieme**: niente ordine, niente duplicati. Le proprietà tra graffe lo cambiano. Per una lista di oggetti basta di solito `{ordered}`; `{nonunique}` si aggiunge quando lo stesso valore può davvero ripetersi, come nei voti (5.2, 5.3).

1.8 Membri statici
sottolineati



Un campo o un metodo `static` appartiene alla classe, non agli oggetti: in UML si **sottolinea**. Una costante `static final` è sottolineata e `{readOnly}`, con il suo valore (1.12). È la firma del Singleton (15.1).

1.9 Classe e operazioni astratte
nome in corsivo

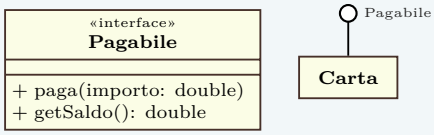


Classe `abstract`: nome in *corsivo* (oppure `{abstract}` accanto al nome, più leggibile a mano). Un metodo `abstract` (senza corpo) è in corsivo; gli altri metodi, già implementati, no.

1.10

Interfaccia

stereotipo «interface»

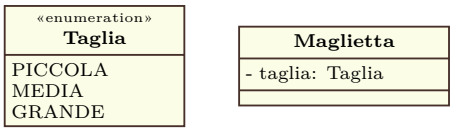


Le operazioni sono tutte pubbliche e astratte; di solito non ci sono attributi. In Java è una `interface`. A destra la forma abbreviata a **pallina** (*lollipop*): la classe `Carta` realizza `Pagabile` (2.9).

1.11

Enumerazione

stereotipo «enumeration»

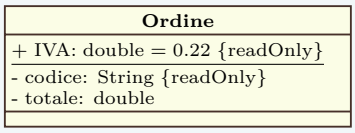


Un elenco fisso di valori, come l'`enum` di Java: nel secondo scomparto i **letterali**. Si usa come tipo degli attributi (`taglia: Taglia`) o si collega con una linea di associazione. Con campi e metodi: 6.5.

1.12

Costanti e sola lettura

{readOnly}



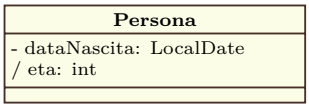
Un campo `final` non cambia dopo la costruzione: `{readOnly}`. Se è anche `static` è una costante della classe, e si sottolinea; il valore si scrive come default (`= 0.22`). Per convenzione le costanti hanno il nome in maiuscolo.

1.13

Attributo derivato

la barra: si calcola da altri

oltre le slide

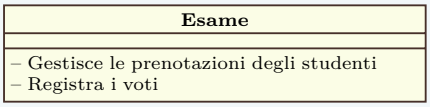


Un valore che non si memorizza ma si **calcola** da altri attributi si scrive con una `/` davanti. Nel codice di solito non c'è il campo, c'è un metodo: `int getEta()` che usa `dataNascita`.

1.14

Responsabilità

nei modelli di dominio, al posto dei metodi

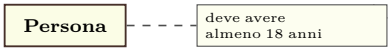


Nel modello del **dominio del problema** non si scrivono metodi (sono un elemento della soluzione), ma **responsabilità**: compiti della classe, preceduti da `--`, nello scomparto delle operazioni.

1.15

Nota

commento collegato con una linea tratteggiata



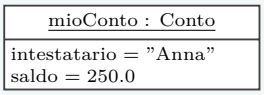
Testo libero su qualunque elemento di qualunque diagramma: è il **commento** di UML. Nei pattern contiene spesso il codice di un metodo (`{ for all o in observers: o.update() }`).

1.16

Oggetto (istanza)

nome sottolineato

oltre le slide



Un oggetto si scrive `nome : Classe` (sottolineato), con i **valori** dei campi. È la notazione dei partecipanti dei diagrammi di sequenza (`:Ordine`) e dei nodi di deployment (`headquarters : TicketServer`).

2 Il diagramma delle classi: le relazioni

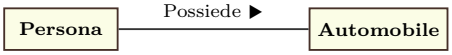
Dispensa: capitolo 7

Le linee e le frecce in un colpo d'occhio

Disegno	Relazione	Si legge	In Java
———	**associazione**	A è collegata a B	un campo (in uno o in entrambi i versi)
———▷	**associazione navigabile**	da A si arriva a B	A ha un campo di tipo B
◇———	**aggregazione**	A (rombo) è fatta di B	campo o collezione di B, parti condivisibili
◆———	**composizione**	B non esiste senza A	B creata e posseduta da A
———▷	**generalizzazione**	A è un B	`class A extends B`
- - - - - -▷	**realizzazione**	A implementa B	`class A implements B`
- - - - - -▷	**dipendenza**	A usa B	parametro, variabile locale, `new B()`, `B.m()`
- - - - - - - - -	**linea tratteggiata**	nota o classe associativa	—

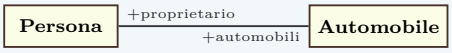
La punta **vuota a triangolo** vuol dire sempre «è un tipo di» (linea continua: eredita, tratteggiata: implementa). Il **rombo** sta sempre dal lato del **tutto**. Il **tratteggio** con la punta aperta è sempre una **dipendenza**.

2.1
Associazione
linea con un nome e il verso di lettura



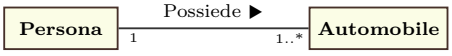
Una relazione fra le istanze di due classi. Il triangolino dice solo come si **legge** il nome: «la Persona possiede l'Automobile». Non dice chi conosce chi (quello è il verso di navigazione, 2.4).

2.2
Ruolo
il nome di un estremo



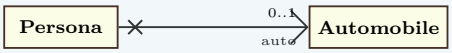
Il ruolo si scrive vicino alla classe che lo ricopre e diventa il **nome del campo** dall'altra parte: `Persona proprietario` sta in `Automobile`, `List<Automobile> automobili` in `Persona`.

2.3
Molteplicità di un'associazione
si scrive vicino alla classe che conta



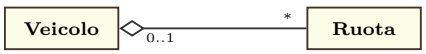
Il `1..*` vicino ad Automobile si legge: «*una* Persona possiede *da 1 a molte* Automobili». Il `1` vicino a Persona: «*un*'Automobile ha *un solo* proprietario». Uno a molti. I costruttori devono rispettarle (4.5).

2.4
Verso di navigazione
freccia aperta; croce = non navigabile



Da una Persona si trova la sua automobile; da un'Automobile **non** si risale alla persona (la croce). In Java: `Automobile auto` sta in `Persona`, e `Automobile` non ha campi di tipo `Persona`. Senza frecce la navigazione non è specificata, oppure va nei due versi. È una scelta di progetto: nei diagrammi concettuali non si mette.

2.5
Aggregazione
rombo vuoto dal lato del tutto



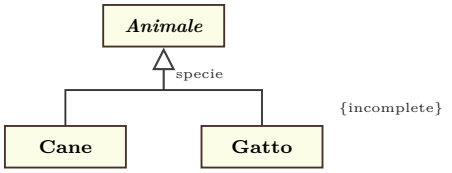
Relazione **tutto–parti**: le parti possono esistere anche da sole (una ruota di scorta) e passare da un tutto all'altro. Se dal lato del tutto non c'è niente, vale `0..1`. Si implementa come un'associazione (7.4).

2.6
Composizione
rombo pieno: le parti muoiono con il tutto



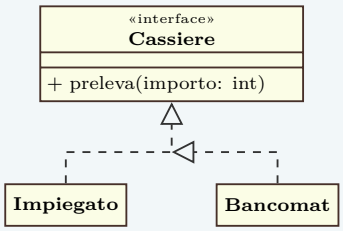
Aggregazione **forte**: una parte appartiene a **un solo** tutto e non esiste senza di lui. Sottintesa la molteplicità `1` dal lato del tutto. Nel codice il tutto crea le parti e non le cede (7.3).

2.7
Generalizzazione
triangolo vuoto verso la superclasse



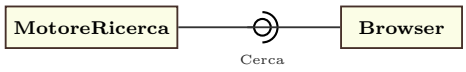
«Cane **è un** Animale»: `class Cane extends Animale`. La sottoclasse eredita attributi e operazioni. Il **discriminatore** (`specie`) dice il criterio; `{incomplete}` (o una linea tratteggiata che taglia le frecce) dice che esistono altre sottoclassi non disegnate.

2.8 Realizzazione
tratteggiata con il triangolo vuoto



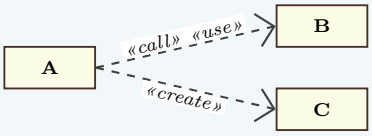
`class Bancomat implements Cassiere`: la classe fornisce il codice di tutte le operazioni dell'interfaccia. Classi di gerarchie diverse possono realizzare la stessa interfaccia, e chi usa un `Cassiere` non sa quale classe c'è dietro.

2.9 Pallina e presa
interfaccia fornita e richiesta



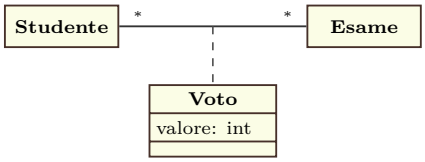
La **pallina** (*lollipop*) è l'interfaccia che `MotoreRicerca` **fornisce** (la implementa); la **presa** (*socket*) è quella che `Browser` **richiede** (la usa: è una dipendenza). Equivale a una realizzazione più una dipendenza verso `«interface» Cerca`.

2.10 Dipendenza
freccia tratteggiata, con uno stereotipo



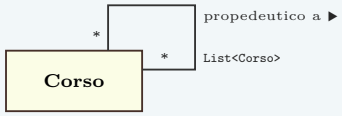
A **usa** B durante l'esecuzione, senza tenerne un riferimento in un campo: **«call»** chiama un suo metodo, **«use»** ne usa un attributo, **«create»** ne crea un'istanza (`new`). Se B cambia, A può non compilare più: è un accoppiamento. Si vede nel progetto di dettaglio (7.1).

2.11 Classe associativa
collegata all'associazione con il tratteggio



Un attributo (il voto) che riguarda la **coppia** studente–esame e non si può dare a nessuno dei due. Tipica dei molti a molti. Equivale a una classe normale con due associazioni uno a molti (`Studente 1 -- * Voto * -- 1 Esame`), che però ammette più voti per la stessa coppia (7.5).

2.12 Associazione riflessiva
una classe collegata a se stessa



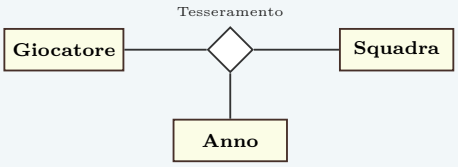
Un `Corso` è propedeutico ad altri corsi: in Java un campo il cui tipo è la classe stessa (`List<Corso> propedeutici`, `Persona coniuge`). Asimmetrica (propedeutico a) o simmetrica (in alternativa a). I ruoli distinguono i due estremi (4.7).

2.13 Associazione qualificata
la chiave per trovare l'oggetto oltre le slide



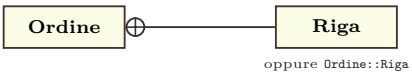
Il rettangolino attaccato a `Rubrica` è il **qualificatore**: dato un nome, si trova al più un contatto. È il disegno di una `Map<String, Contatto>` (5.7). La molteplicità vale **per ogni chiave**.

2.14 Associazione n-aria
tre o più classi, con un rombo oltre le slide



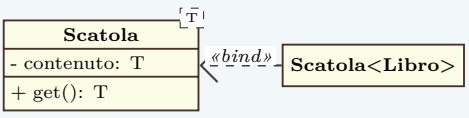
Un legame fra **tre** oggetti insieme (un giocatore, in una squadra, in un anno). Il rombo vuoto **non** è un'aggregazione: è il punto in cui si incontrano le linee. In Java diventa una classe con tre riferimenti, come una classe associativa.

2.15 Classe annidata
cerchio con la croce dal lato della classe esterna
oltre le slide



Una classe dichiarata **dentro** un'altra (`class Ordine { class Riga {} }`): la linea con il cerchio e la croce. In alternativa si scrive il nome completo `Ordine::Riga`. Non dice nulla sui campi: se Ordine contiene righe, serve anche l'associazione (6.7).

2.16 Classe template
parametro in un rettangolo tratteggiato oltre le slide



Una classe generica `class Scatola<T>`: il parametro `T` sta nel rettangolo tratteggiato in alto a destra. Una versione con un tipo concreto (`Scatola<Libro>`) si collega con «bind» ($T \to$ `Libro`), oppure si scrive solo come tipo di un attributo (6.6).

Due errori di lettura frequenti

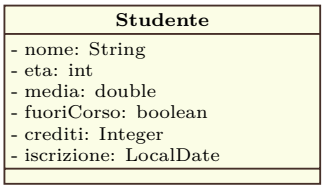
- Il **triangolino** accanto al nome (▶) dice come si legge; la **freccia** sulla linea dice chi conosce chi. Un diagramma concettuale ha spesso il primo e quasi mai la seconda.
- La molteplicità scritta vicino a una classe conta le istanze di **quella** classe collegate a **una** istanza dell'**altra**. Per leggerla si parte sempre dalla classe opposta.

3 Dal codice al diagramma: attributi e operazioni

Dispensa: capitolo 7. A sinistra il codice, a destra il diagramma

3.1 Campi di tipo primitivo, String, wrapper e di libreria

```java
class Studente {
    private String nome;
    private int eta;
    private double media;
    private boolean fuoriCorso;
    private Integer crediti;     // come int
    private LocalDate iscrizione; // di libreria
}
```

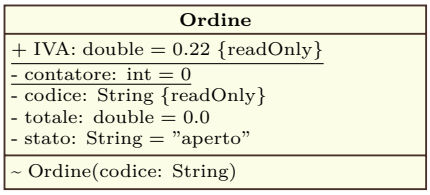


Ogni campo diventa un attributo `visibilità nome: Tipo`: il tipo va **dopo** i due punti, al contrario di Java. Primitivi, wrapper (`Integer`), `String` e classi di libreria (`LocalDate`) restano sempre attributi.

3.2 Valori iniziali, campi final, costanti

```java
class Ordine {
    public static final double IVA = 0.22;
    private static int contatore = 0;
    private final String codice;
    private double totale = 0.0;
    private String stato = "aperto";

    Ordine(String codice) {
        this.codice = codice;
        contatore++;
    }
}
```

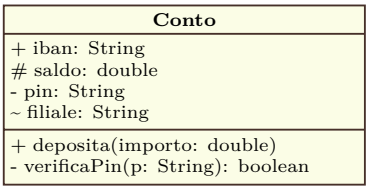


L'inizializzazione diventa `= valore`. `final` diventa `{readOnly}`. `static` si **sottolinea**: `static final` è una costante della classe. Il costruttore senza modificatore ha visibilità di package, `~`.

3.3 Le quattro visibilità

```java
public class Conto {
    public String iban;
    protected double saldo;
    private String pin;
    String filiale;              // package

    public void deposita(double importo) {
        saldo += importo;
    }
    private boolean verificaPin(String p) {
        return pin.equals(p);
    }
}
```

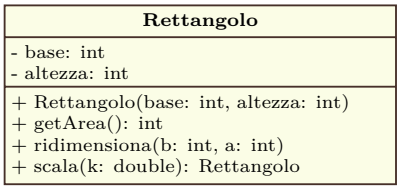


`public` → `+`, `protected` → `#`, `private` → `-`, nessun modificatore (package) → `~`. Valgono allo stesso modo per attributi e operazioni.

3.4 Costruttori e metodi: la firma

```java
public class Rettangolo {
    private int base, altezza;

    public Rettangolo(int base, int altezza) {
        this.base = base;
        this.altezza = altezza;
    }
    public int getArea() { return base * altezza; }
    public void ridimensiona(int b, int a) {
        base = b;
        altezza = a;
    }
    public Rettangolo scala(double k) {
        return new Rettangolo((int) (base * k),
                              (int) (altezza * k));
    }
}
```

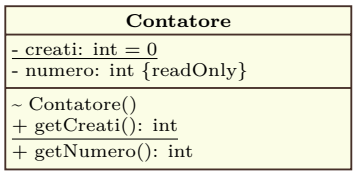


Parametri `nome: tipo`, separati da virgole; il tipo restituito dopo la parentesi e i due punti; `void` si omette. Il costruttore ha il nome della classe e nessun tipo. `int base, altezza;` sono **due** attributi.

3.5 Campi e metodi statici

```java
class Contatore {
    private static int creati = 0;
    private final int numero;

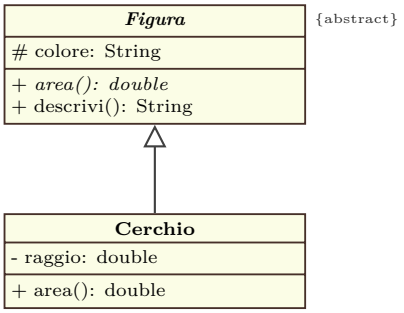
    Contatore() {
        creati++;
        numero = creati;
    }
    public static int getCreati() {
        return creati;
    }
    public int getNumero() { return numero; }
}
```



Tutto ciò che è `static` si sottolinea: `creati` è uno solo per tutta la classe, `numero` uno per ogni oggetto. Un metodo statico si chiama sulla classe: `Contatore.getCreati()`.

3.6 Classe astratta e metodi astratti

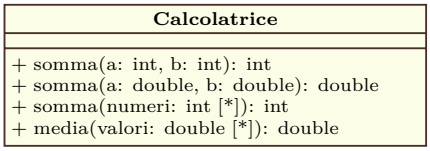
```java
abstract class Figura {
    protected String colore;
    public abstract double area();
    public String descrivi() {
        return colore + ", area " + area();
    }
}
class Cerchio extends Figura {
    private double raggio;
    @Override
    public double area() {
        return Math.PI * raggio * raggio;
    }
}
```



`abstract` → *corsivo* (o `{abstract}`): la classe e i metodi senza corpo. La sottoclasse ripete solo i metodi che **ridefinisce** (`@Override`), non quelli ereditati (`colore`, `descrivi`).

3.7 Overloading, parametri array e varargs

```java
class Calcolatrice {
    public int somma(int a, int b) { return a + b; }
    public double somma(double a, double b) {
        return a + b;
    }
    public int somma(int... numeri) {
        int s = 0;
        for (int n : numeri) s += n;
        return s;
    }
    public double media(double[] valori) {
        double s = 0;
        for (double v : valori) s += v;
        return s / valori.length;
    }
}
```

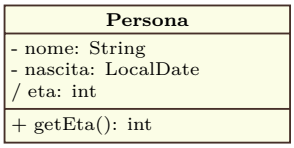


Un'operazione per ogni metodo, anche con lo stesso nome (*overloading*). Un parametro array o *varargs* (`int...`) ha la molteplicità `[*]` oltre le slide. Lo scomparto degli attributi resta vuoto, ma si disegna.

3.8 Getter, setter e valori calcolati

```java
class Persona {
    private String nome;
    private LocalDate nascita;

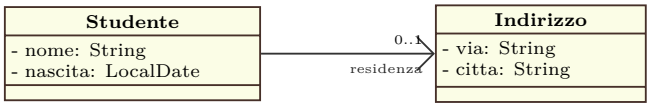
    public String getNome() { return nome; }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public int getEta() {      // nessun campo eta
        return Period.between(nascita,
                LocalDate.now()).getYears();
    }
}
```



Getter e setter di solito **non** si disegnano: dicono solo che l'attributo si legge e si scrive (si possono mettere se il testo lo chiede). Un `get` senza un campo corrispondente è un **attributo derivato** `/eta` oltre le slide, oppure resta un'operazione.

3.9 Attributo o associazione? Il tipo decide

```java
class Studente {
    private String nome;          // di libreria
    private LocalDate nascita;    // di libreria
    private Indirizzo residenza; // del diagramma
}
class Indirizzo {
    private String via;
    private String citta;
}
```



Se il tipo del campo è una **classe del diagramma**, il campo non va fra gli attributi: diventa un'**associazione**, e il suo nome diventa il **ruolo**. `0..1` perché il campo può restare `null`. Scrivere `residenza: Indirizzo` fra gli attributi nasconde la relazione; scriverlo **e** disegnare la freccia è un doppione.

4 Dal codice al diagramma: riferimenti fra classi

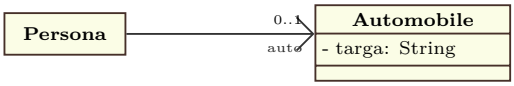
Dispensa: capitolo 7. Un campo il cui tipo è una classe del diagramma diventa un'associazione

Come si legge un campo che punta a un'altra classe

1. Il campo `T nome` nella classe `A` diventa una linea da `A` a `T`, con la **freccia** verso `T` (da `A` si arriva a `T`). Se anche `T` ha un campo verso `A`, la linea è una sola e **senza frecce**.
2. Il **nome del campo** è il **ruolo**: si scrive vicino a `T`.
3. La **molteplicità vicino a `T`** viene dal campo: `0..1` se può essere `null`, `1` se è sempre impostato (costruttore, `final`), `*`, `1..*`, `0..4`… se è una collezione (argomento 5).
4. La **molteplicità vicino a `A`** (quanti `A` per uno stesso `T`) nel codice di `A` non c'è: viene dal campo inverso, se esiste, o dal testo dell'esercizio. Se nessuno la dice, si lascia vuota.

4.1 Un riferimento che può mancare

```
class Persona {
    private Automobile auto;  // null se non compra
    void compra(Automobile a) { auto = a; }
}
class Automobile {
    private String targa;
}
```

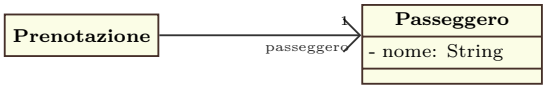


Freccia da chi ha il campo verso il tipo del campo; ruolo `auto`; `0..1` perché `auto` può valere `null`. `Automobile` non ha campi verso `Persona`: non è navigabile all'indietro (si può aggiungere la croce, 2.4).

4.2 Un riferimento obbligatorio

```
class Prenotazione {
    private final Passeggero passeggero;

    Prenotazione(Passeggero p) {
        if (p == null)
            throw new IllegalArgumentException();
        passeggero = p;
    }
}
class Passeggero {
    private String nome;
}
```

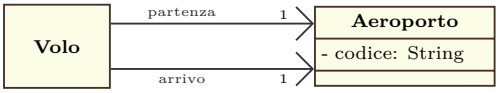


Il campo è `final`, assegnato nel costruttore e mai `null`: molteplicità `1`. Non può esistere una prenotazione senza passeggero, e il costruttore è il punto in cui il codice lo garantisce.

4.3 Due campi dello stesso tipo

```
class Volo {
    private final Aeroporto partenza;
    private final Aeroporto arrivo;

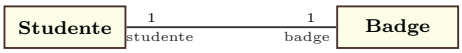
    Volo(Aeroporto da, Aeroporto a) {
        partenza = da;
        arrivo = a;
    }
}
class Aeroporto {
    private String codice;
}
```



Due campi, **due** associazioni, ciascuna con il suo ruolo. Non una sola linea con molteplicità 2: i due aeroporti hanno significati diversi.

4.4 Uno a uno, nei due versi

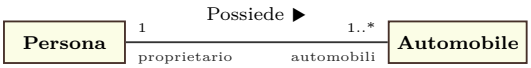
```
class Studente {
    private final Badge badge;
    Studente() {
        badge = new Badge(this);   // crea il badge
    }
}
class Badge {
    private final Studente studente;
    Badge(Studente s) {        // riceve lo studente
        studente = s;
    }
}
```



Un campo in **entrambe** le classi: una sola linea, **senza frecce**, con un ruolo e una molteplicità a ogni estremo. `1`–`1`: un lato crea l'altro e gli passa `this`. Se ognuno creasse l'altro (`new Badge()` che fa `new Studente()`…) la ricorsione non finirebbe mai (`StackOverflowError`).

4.5 Uno a molti, nei due versi

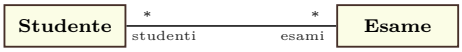
```
class Persona {
    private final List<Automobile> automobili =
            new ArrayList<>();
    Persona() {                // almeno un'auto
        automobili.add(new Automobile(this));
    }
    void compraNuova() {
        automobili.add(new Automobile(this));
    }
}
class Automobile {
    private final Persona proprietario;
    Automobile(Persona p) { proprietario = p; }
}
```



La **lista** sta dal lato «uno» (`Persona`), il riferimento singolo dal lato «molti» (`Automobile`). `1..*`: il costruttore di `Persona` crea subito un'automobile. `1`: ogni automobile riceve il proprietario nel costruttore e non lo cambia più.

4.6 Molti a molti

```
class Studente {
    private final List<Esame> esami =
            new ArrayList<>();
    void supera(Esame e) {
        esami.add(e);
        e.registra(this);   // allinea l'altro lato
    }
}
class Esame {
    private final List<Studente> studenti =
            new ArrayList<>();
    void registra(Studente s) { studenti.add(s); }
}
```

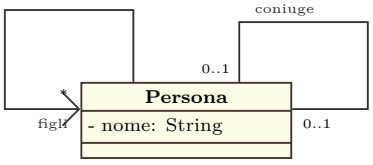


Una collezione in **ciascuna** classe: `*` da entrambe le parti, senza frecce. I metodi devono tenere allineati i due lati. Se serve un dato sulla coppia (il voto) ci vuole una classe associativa (7.5).

4.7 Un campo del tipo della classe stessa

```
class Persona {
    private String nome;
    private Persona coniuge;                  // 0..1
    private final List<Persona> figli =
            new ArrayList<>();                // *

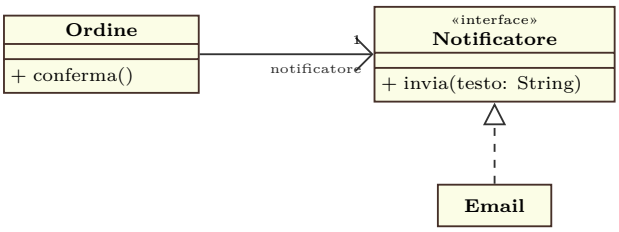
    void sposa(Persona p) {
        coniuge = p;
        p.coniuge = this;
    }
    void nasce(Persona f) { figli.add(f); }
}
```



Un'associazione dalla classe a se stessa (**riflessiva**). `coniuge` va nei due versi (`sposa` aggiorna entrambi): linea senza frecce, `0..1` da ogni lato. `figli` si naviga solo dal genitore: freccia, `*`.

4.8 Un campo di tipo interfaccia

```
interface Notificatore {
    void invia(String testo);
}
class Email implements Notificatore {
    public void invia(String testo) { /* ... */ }
}
class Ordine {
    private final Notificatore notificatore;
    Ordine(Notificatore n) { notificatore = n; }
    void conferma() {
        notificatore.invia("Ordine confermato");
    }
}
```

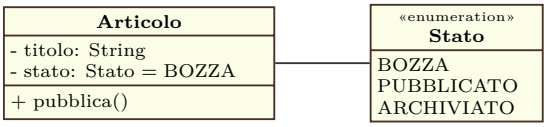


La freccia va all'**interfaccia**, il tipo del campo, non alla classe concreta: `Ordine` non sa nulla di `Email`. L'oggetto concreto arriva dal costruttore. È la forma della Dependency Injection e dello Strategy (7.6).

4.9 Un campo di tipo enumerazione

```
enum Stato { BOZZA, PUBBLICATO, ARCHIVIATO }

class Articolo {
    private String titolo;
    private Stato stato = Stato.BOZZA;
    void pubblica() { stato = Stato.PUBBLICATO; }
}
```



Il campo di tipo `enum` resta un **attributo**, con il default (`= BOZZA`); l'enumerazione si disegna a parte con i suoi letterali. La linea fra le due è facoltativa, di solito senza ruoli né molteplicità.

5 Dal codice al diagramma: array e collezioni

Dispensa: capitolo 7. Le collezioni diventano molteplicità

Array e collezioni in un colpo d'occhio

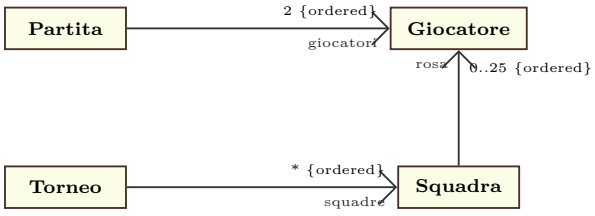
Se gli elementi sono oggetti di una **classe del diagramma**, la collezione è un'**associazione** e la molteplicità si scrive vicino a quella classe. Se sono valori (`int`, `String`, `LocalDate`…) è un **attributo**, con la molteplicità fra quadre: `voti: int [*] {ordered, nonunique}`.

Nel codice	Molteplicità e proprietà	Caso
`B[] x = new B[n]`	`n` se è sempre pieno, `0..n` se no; `{ordered}`	5.1
`B[] x` di lunghezza decisa a runtime	`* {ordered}`	5.1
`int[] x = new int[n]`, `String[] x`	attributo `x: int [n]`, `x: String [*]`, `{ordered}`	5.2
`List<B>`, `ArrayList<B>`, `LinkedList<B>`	`* {ordered}`, più `{nonunique}` se ammette doppioni	5.3
`Set<B>`, `HashSet<B>`	`*` e basta: è il significato predefinito	5.5
`TreeSet<B>`, `LinkedHashSet<B>`, `SortedSet<B>`	`* {ordered}`	5.5
`Queue<B>`, `Deque<B>`, `Stack<B>`	`* {ordered}` oltre le slide	5.6
`Collection<B>`, `Iterable<B>`	`*`	5.6
`Map<K, B>`	qualificatore `k: K`, poi `0..1` oltre le slide	5.7
`Map<K, List<B>>`	qualificatore `k: K`, poi `* {ordered}` oltre le slide	5.8
`Optional<B>`	`0..1` oltre le slide	5.10
`B[][]`	qualificatori `i: int, j: int`, poi `0..1` oltre le slide	5.11
almeno uno, al più n (costruttore, controlli)	`1..*`, `0..n`, `1..n`	5.12

La classe concreta (`ArrayList`, `HashMap`…) **non** compare nel diagramma: è un dettaglio di implementazione. E `final` su una collezione **non** è `{readOnly}`: il riferimento non cambia, ma gli elementi sì.

5.1 Array di oggetti

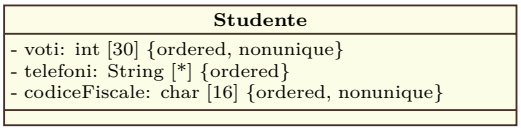
```
class Partita {
    private final Giocatore[] giocatori =
            new Giocatore[2];
    Partita(Giocatore a, Giocatore b) {
        giocatori[0] = a;     // sempre pieno
        giocatori[1] = b;
    }
}
class Squadra {
    private final Giocatore[] rosa =
            new Giocatore[25];
    private int iscritti = 0;
    void iscrivi(Giocatore g) {
        rosa[iscritti++] = g;   // man mano
    }
}
class Torneo {
    private final Squadra[] squadre;
    Torneo(int n) { squadre = new Squadra[n]; }
}
class Giocatore { }
```



Un array di oggetti è un'associazione, e la molteplicità è la **lunghezza**: `2` se le celle sono sempre tutte piene, `0..25` se si riempiono man mano (le altre restano `null`), `*` se la lunghezza si decide durante l'esecuzione. `{ordered}`: ogni elemento ha un indice.

5.2 Array di valori: primitivi e String

```
class Studente {
    private int[] voti = new int[30];
    private String[] telefoni;
    private char[] codiceFiscale = new char[16];
}
```

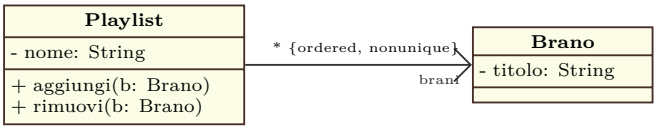


Un array di valori è un **attributo** con la molteplicità fra quadre: la lunghezza se è fissa (`[30]`), `[*]` se non si conosce. `{ordered}` perché le celle hanno un indice; `{nonunique}` perché lo stesso voto, o la stessa lettera, può ripetersi. Il tipo Java (`voti: int[]`) si capisce, ma non è la notazione UML.

5.3 Liste di oggetti: List, ArrayList, LinkedList

```
class Playlist {
    private String nome;
    private final List<Brano> brani =
            new ArrayList<>();

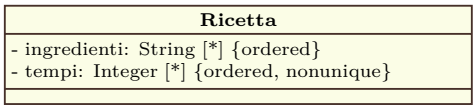
    void aggiungi(Brano b) { brani.add(b); }
    void rimuovi(Brano b) { brani.remove(b); }
}
class Brano {
    private String titolo;
}
```



`List`, `ArrayList`, `LinkedList`, `Vector`: associazione `* {ordered}`; il nome della lista è il ruolo. Qui anche `{nonunique}`, perché lo stesso brano può comparire due volte; in una lista dove ogni oggetto c'è una volta sola basta `{ordered}`. Con `new ArrayList` o `new LinkedList` il diagramma non cambia.

5.4 Liste di valori

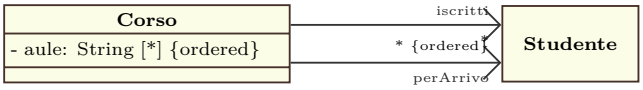
```
class Ricetta {
    private final List<String> ingredienti =
            new ArrayList<>();
    private final List<Integer> tempi =
            new LinkedList<>();
}
```



Una collezione di `String`, `Integer` o altri tipi di libreria è un **attributo** con molteplicità, non un'associazione (non c'è una classe `String` nel diagramma). `final` non diventa `{readOnly}`: la lista è sempre la stessa, ma il suo contenuto cambia.

5.5 Insiemi: Set, HashSet, TreeSet, LinkedHashSet

```
class Corso {
    private final Set<Studente> iscritti =
            new HashSet<>();
    private final Set<Studente> perArrivo =
            new LinkedHashSet<>(); // per arrivo
    private final Set<String> aule =
            new TreeSet<>();       // alfabetico
}
class Studente { }
```



`Set` e `HashSet`: `*` senza proprietà, perché per UML una molteplicità è già un insieme (niente doppioni, niente ordine). `LinkedHashSet` (ordine di inserimento) e `TreeSet` (ordine naturale): `{ordered}`. In un insieme i doppioni non ci sono mai, quindi mai `{nonunique}`.

5.6 Code, pile e collezioni generiche oltre le slide

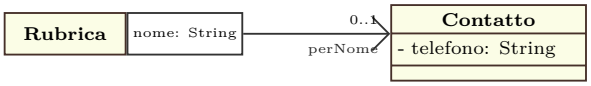
```
class Sportello {
    private final Queue<Cliente> inAttesa =
            new ArrayDeque<>();  // coda: FIFO
    private final Deque<Pratica> daFirmare =
            new ArrayDeque<>();  // pila: LIFO
    private final Collection<Cliente> serviti =
            new ArrayList<>();
}
class Cliente { }
class Pratica { }
```



`Queue`, `Deque`, `Stack`: `* {ordered}`, perché chi esce dipende dall'ordine (FIFO o LIFO: lo si può scrivere in una nota). `Collection` e `Iterable` non promettono né ordine né unicità: `*` e basta.

5.7 Mappe da una chiave a un oggetto oltre le slide

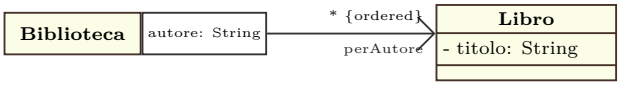
```
class Rubrica {
    private final Map<String, Contatto> perNome =
            new HashMap<>();
    void aggiungi(String nome, Contatto c) {
        perNome.put(nome, c);
    }
    Contatto cerca(String nome) {
        return perNome.get(nome);
    }
}
class Contatto {
    private String telefono;
}
```



`Map<K, V>` con `V` classe del diagramma: **associazione qualificata**. Il qualificatore è la chiave (`nome: String`); `0..1` vale **per ogni chiave**: dato un nome, al più un contatto. Se il qualificatore non serve, va bene anche un'associazione `*` con il ruolo `perNome`.

5.8 Mappe da una chiave a una lista oltre le slide

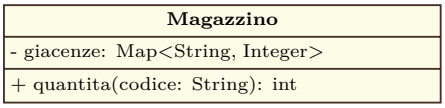
```
class Biblioteca {
    private final Map<String, List<Libro>>
            perAutore = new HashMap<>();
    List<Libro> diAutore(String autore) {
        return perAutore.getOrDefault(autore,
                                      List.of());
    }
}
class Libro {
    private String titolo;
}
```



La chiave seleziona un **gruppo**: dopo il qualificatore la molteplicità è quella della lista, `* {ordered}`. Con `Map<String, Set<Libro>>` sarebbe `*` senza proprietà.

5.9 Mappe di valori oltre le slide

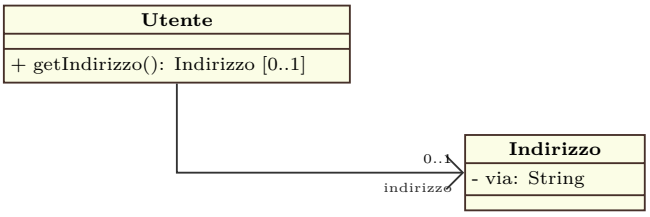
```
class Magazzino {
    private final Map<String, Integer> giacenze =
            new HashMap<>();
    int quantita(String codice) {
        return giacenze.getOrDefault(codice, 0);
    }
}
```



Se chiave e valore sono tipi di libreria, la mappa è un **attributo** con il suo tipo Java. Se la coppia ha un significato nel dominio, è meglio una classe (`Giacenza`, con `codice` e `quantita`) e un'associazione `*` verso di lei.

5.10 Optional oltre le slide

```
class Utente {
    private Indirizzo indirizzo;   // può mancare
    Optional<Indirizzo> getIndirizzo() {
        return Optional.ofNullable(indirizzo);
    }
}
class Indirizzo {
    private String via;
}
```

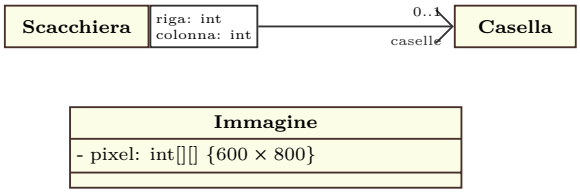


`Optional<T>` vuol dire «zero o uno»: `0..1`. Come tipo di ritorno si scrive `T [0..1]`; come campo, `Optional<Indirizzo>` si disegna come un `Indirizzo` che può essere `null`, cioè un'associazione `0..1`.

5.11 Matrici oltre le slide

```
class Scacchiera {
    private final Casella[][] caselle =
            new Casella[8][8];
    Casella casella(int riga, int colonna) {
        return caselle[riga][colonna];
    }
}
class Casella { }

class Immagine {
    private int[][] pixel = new int[600][800];
}
```

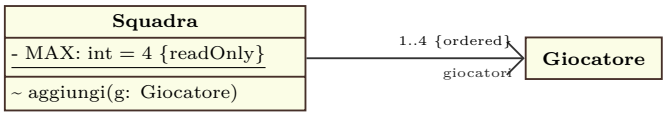


UML non ha molteplicità a due dimensioni. Per una matrice di **oggetti** si usa un'associazione qualificata con **due** qualificatori: data riga e colonna, al più una casella (`1` se è sempre piena). Per una matrice di **valori** si scrive il tipo Java, con le dimensioni in una nota o fra graffe; in alternativa la riga diventa una classe.

5.12 Almeno uno, al più quattro: i vincoli del codice

```
class Squadra {
    private static final int MAX = 4;
    private final List<Giocatore> giocatori =
            new ArrayList<>();

    Squadra(Giocatore capitano) {   // almeno uno
        giocatori.add(capitano);
    }
    void aggiungi(Giocatore g) {   // al più 4
        if (giocatori.size() == MAX)
            throw new IllegalStateException();
        giocatori.add(g);
    }
}
class Giocatore { }
```



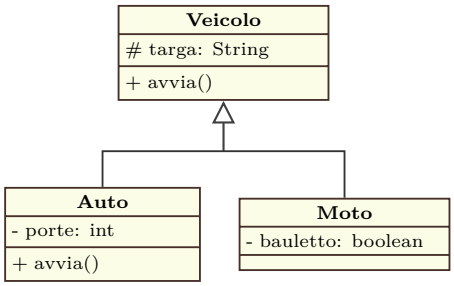
La molteplicità viene da ciò che il codice **garantisce**: il costruttore mette subito un giocatore (minimo `1`), il controllo in `aggiungi` impedisce il quinto (massimo `4`). Nel testo dell'esercizio la stessa informazione è «squadre di al più 4 giocatori, con un capitano».

6 Dal codice al diagramma: ereditarietà, interfacce, tipi speciali

Dispensa: capitolo 7. extends, implements, enum, e i costrutti che le slide non mostrano

6.1 extends: la generalizzazione

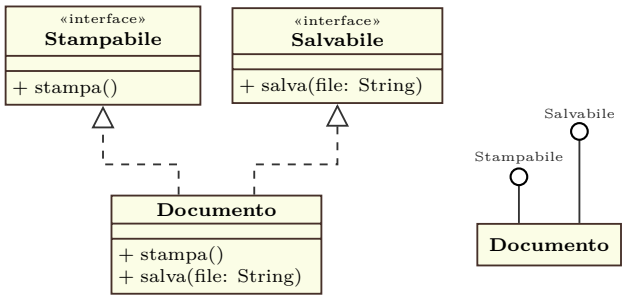
```
class Veicolo {
    protected String targa;
    public void avvia() { }
}
class Auto extends Veicolo {
    private int porte;
    @Override
    public void avvia() { }   // ridefinito
}
class Moto extends Veicolo {
    private boolean bauletto;
}
```



Una freccia a triangolo vuoto da ogni sottoclasse alla superclasse (le punte si possono unire in una sola). La sottoclasse mostra solo ciò che **aggiunge** (`porte`) e ciò che **ridefinisce** (`avvia`); `targa` è ereditato e non si ripete. `protected` → `#`.

6.2 implements: la realizzazione, anche di più interfacce

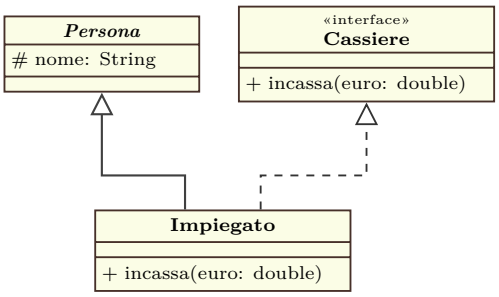
```
interface Stampabile {
    void stampa();
}
interface Salvabile {
    void salva(String file);
}
class Documento implements Stampabile, Salvabile {
    public void stampa() { }
    public void salva(String file) { }
}
```



Freccia **tratteggiata** con il triangolo vuoto verso ogni interfaccia implementata; la classe elenca i metodi che implementa. A destra la forma abbreviata a palline (2.9): stesso significato.

6.3 extends e implements insieme

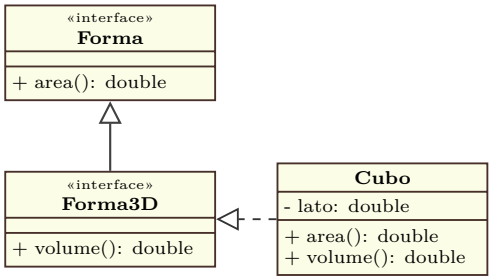
```
abstract class Persona {
    protected String nome;
}
interface Cassiere {
    void incassa(double euro);
}
class Impiegato extends Persona
                implements Cassiere {
    public void incassa(double euro) { }
}
```



Una sola superclasse (linea continua) e quante interfacce si vuole (tratteggiate): in Java si eredita da **una** classe sola. Classe astratta in corsivo.

6.4 Un'interfaccia che ne estende un'altra

```
interface Forma {
    double area();
}
interface Forma3D extends Forma {
    double volume();
}
class Cubo implements Forma3D {
    private double lato;
    public double area() { return 6 * lato * lato; }
    public double volume() {
        return lato * lato * lato;
    }
}
```

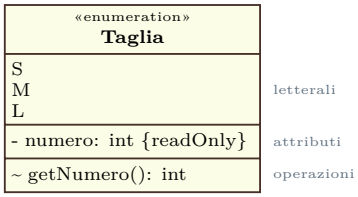


Fra due interfacce `extends` è una **generalizzazione**: linea **continua**. Fra classe e interfaccia `implements` è una **realizzazione**: linea **tratteggiata**. Il `Cubo` deve implementare i metodi di entrambe.

6.5 Enumerazioni con campi e metodi oltre le slide

```
enum Taglia {
    S(44), M(48), L(52);

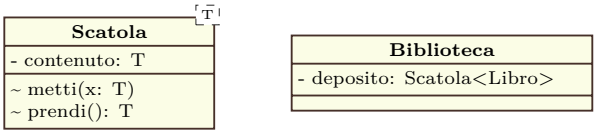
    private final int numero;
    Taglia(int numero) { this.numero = numero; }
    int getNumero() { return numero; }
}
```



Un'`enum` di Java può avere campi, costruttore e metodi: l'enumerazione UML ha allora uno scomparto per i **letterali** e, sotto, quelli soliti per attributi e operazioni. Il costruttore (sempre privato) di solito non si disegna.

6.6 Classi generiche oltre le slide

```
class Scatola<T> {
    private T contenuto;
    void metti(T x) { contenuto = x; }
    T prendi() { return contenuto; }
}
class Libro { }
class Biblioteca {
    private final Scatola<Libro> deposito =
            new Scatola<>();
}
```

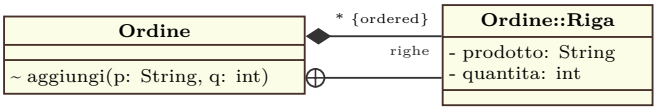


Il parametro di tipo `T` va nel rettangolo **tratteggiato** in alto a destra; dentro la classe `T` si usa come un tipo qualunque. Dove la classe si usa con un tipo concreto basta scrivere `Scatola<Libro>` come tipo dell'attributo (oppure `«bind»`, 2.16).

6.7 Classi annidate e interne oltre le slide

```
class Ordine {
    private final List<Riga> righe =
            new ArrayList<>();

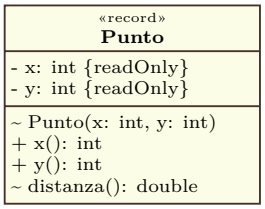
    class Riga {                // classe interna
        private String prodotto;
        private int quantita;
    }
    void aggiungi(String p, int q) {
        Riga r = new Riga();
        r.prodotto = p;
        r.quantita = q;
        righe.add(r);
    }
}
```



Che `Riga` sia dichiarata **dentro** `Ordine` si mostra con il cerchio con la croce (oppure con il nome `Ordine::Riga`). È un'informazione sul codice, non sui legami fra oggetti: la lista `righe` va disegnata a parte, qui come **composizione** (le righe nascono dentro l'ordine e non escono).

6.8 Record oltre le slide

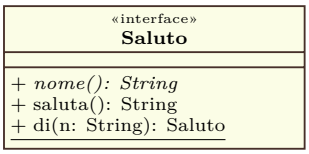
```java
record Punto(int x, int y) {
    double distanza() {
        return Math.sqrt(x * x + y * y);
    }
}
```



Un `record` è una classe `final` con un campo `private final` per ogni componente, un costruttore con tutti i componenti (con la visibilità del record: qui di package) e un metodo di accesso `public` per ciascuno (`x()`, `y()`): attributi `{readOnly}`. Lo stereotipo «record» non è standard ma si capisce; metodi d'accesso e costruttore si possono omettere.

6.9 Interfacce con metodi default e statici oltre le slide

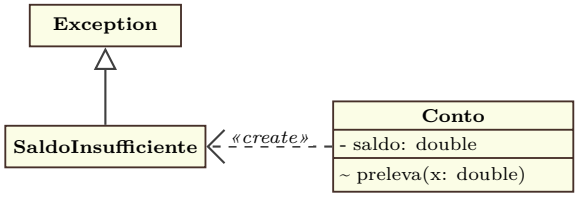
```java
interface Saluto {
    String nome();                    // astratto
    default String saluta() {         // con corpo
        return "Ciao " + nome();
    }
    static Saluto di(String n) {      // statico
        return () -> n;
    }
}
```



In un'interfaccia le operazioni sono astratte e di solito non si scrivono in corsivo; se ci sono metodi `default`, il corsivo distingue quelli **senza** corpo. Il metodo `static` si sottolinea. Nelle interfacce tutto è `public`.

6.10 Eccezioni personalizzate oltre le slide

```java
class SaldoInsufficiente extends Exception {
    SaldoInsufficiente(String msg) { super(msg); }
}
class Conto {
    private double saldo;
    void preleva(double x)
            throws SaldoInsufficiente {
        if (x > saldo)
            throw new SaldoInsufficiente("poco");
        saldo -= x;
    }
}
```



Un'eccezione è una classe come le altre: `extends Exception` è una generalizzazione verso la classe di libreria (si disegna solo il nome). `Conto` la crea con `throw new`: dipendenza «create». La clausola `throws` non ha un simbolo nelle slide; se serve, si scrive in una nota.

7 Dal codice al diagramma: dipendenze e tutto-parti

Dispensa: capitolo 7. Che cosa non è un campo, e quando un campo è una parte

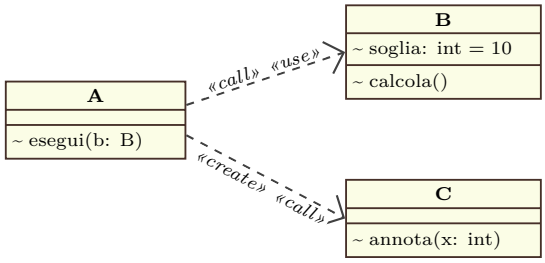
Associazione, dipendenza, aggregazione o composizione?

1. La classe B compare **solo dentro un metodo** di A (parametro, variabile locale, tipo restituito, `new B()`, `B.metodoStatico()`)? **Dipendenza** tratteggiata, con «call», «use» o «create».
2. B è il tipo di un **campo** di A? **Associazione** (argomenti 4 e 5). Poi ci si chiede se B è **parte** di A («è fatto di», «contiene», «è composto da»):
3. no → resta un'**associazione**; sì, ma le parti arrivano da fuori, si possono togliere e sopravvivono → **aggregazione** (rombo vuoto); sì, e A crea le parti, non le cede e le parti non esistono senza A → **composizione** (rombo pieno).

Nel codice aggregazione e composizione si scrivono quasi come un'associazione: la differenza la fanno **chi crea** le parti e **se le parti escono**.

7.1 Le tre dipendenze: «call», «use», «create»

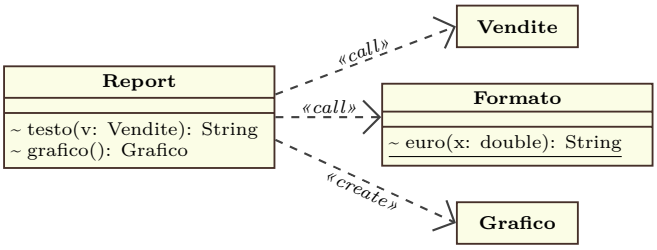
```
class A {
    void esegui(B b) {
        b.calcola();              // «call»
        int limite = b.soglia;    // «use»
        C registro = new C();     // «create»
        registro.annota(limite);
    }
}
class B {
    int soglia = 10;
    void calcola() { }
}
class C {
    void annota(int x) { }
}
```



A non ha campi di tipo B o C: niente associazioni, solo **dipendenze**, perché li usa dentro un metodo. Si chiama un metodo: «call»; si legge un attributo: «use»; si crea un oggetto: «create». Più stereotipi sulla stessa freccia si possono scrivere insieme. Se B o C cambiano, A può non compilare più.

7.2 Parametri, valori restituiti, metodi statici

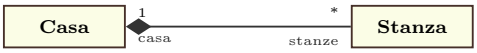
```
class Report {
    String testo(Vendite v) {          // parametro
        return Formato.euro(v.totale()); // statico
    }
    Grafico grafico() {        // tipo restituito
        return new Grafico();
    }
}
class Vendite {
    double totale() { return 0; }
}
class Formato {
    static String euro(double x) {
        return x + " euro";
    }
}
class Grafico { }
```



Una classe che compare come **tipo di un parametro** o **del valore restituito**, o di cui si chiama un **metodo statico** (`Formato.euro`), è una dipendenza: non c'è un campo che la tenga. Il tipo del parametro resta comunque nella firma dell'operazione.

7.3 Composizione: il tutto crea le parti e non le cede

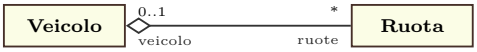
```
class Casa {
    private final List<Stanza> stanze =
            new ArrayList<>();
    Casa(int quante) {
        for (int i = 0; i < quante; i++)
            stanze.add(new Stanza(this));
    }
}
class Stanza {
    private final Casa casa;
    Stanza(Casa c) { casa = c; } // dentro una casa
}
```



Gli indizi della **composizione**: le parti sono create **dentro** il tutto (`new Stanza` nel costruttore di `Casa`), stanno in un campo privato che non esce mai (niente getter della lista, niente `add` dall'esterno), e ricevono il tutto nel costruttore, `final`. Rombo pieno dal lato di `Casa`, molteplicità `1`.

7.4 Aggregazione: le parti arrivano da fuori e sopravvivono

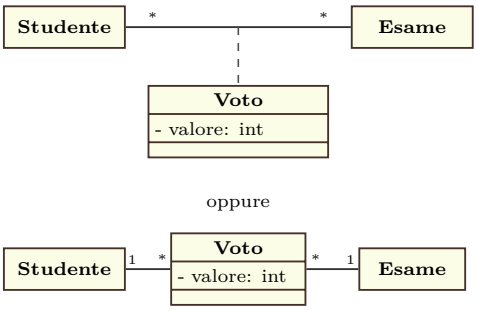
```
class Veicolo {
    private final List<Ruota> ruote =
            new ArrayList<>();
    void monta(Ruota r) {
        ruote.add(r);
        r.setVeicolo(this);
    }
    Ruota smonta(int i) {
        Ruota r = ruote.remove(i);
        r.setVeicolo(null);
        return r;          // la ruota esiste ancora
    }
}
class Ruota {
    private Veicolo veicolo;   // null: di scorta
    void setVeicolo(Veicolo v) { veicolo = v; }
}
```



Le ruote sono create **fuori** e passate a `monta`; `smonta` le restituisce, e possono finire su un altro veicolo o restare di scorta (`null`): **aggregazione**, `0..1` dal lato del tutto. Se le ruote non fossero «parti» del veicolo (per esempio i passeggeri), sarebbe una semplice associazione.

7.5 Un dato sulla coppia: classe associativa

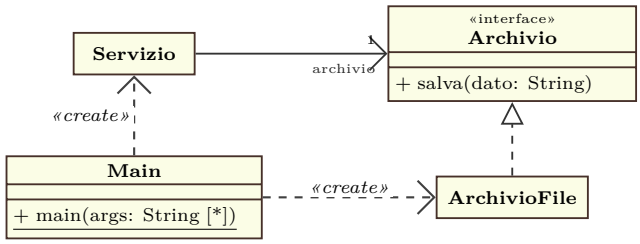
```
class Voto {
    private final Studente studente;
    private final Esame esame;
    private final int valore;
    Voto(Studente s, Esame e, int valore) {
        studente = s;
        esame = e;
        this.valore = valore;
    }
}
class Studente {
    private final List<Voto> voti =
            new ArrayList<>();
}
class Esame {
    private final List<Voto> voti =
            new ArrayList<>();
}
```



Una classe con un riferimento a **due** classi più un dato che riguarda la coppia: **classe associativa** sul molti a molti (sopra), oppure due uno a molti (sotto). La seconda forma ammette più voti per la stessa coppia studente–esame (un esame ripetuto); la prima no.

7.6 La dipendenza passata da fuori (Dependency Injection)

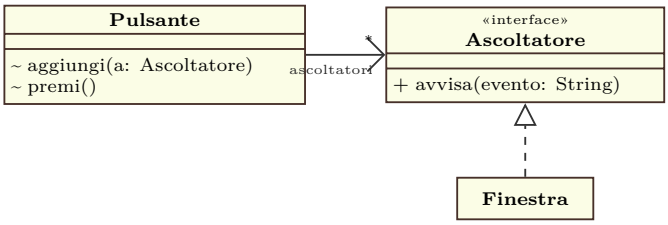
```
interface Archivio {
    void salva(String dato);
}
class ArchivioFile implements Archivio {
    public void salva(String dato) { }
}
class Servizio {
    private final Archivio archivio;
    Servizio(Archivio a) {     // iniettato da fuori
        archivio = a;
    }
}
public class Main {
    public static void main(String[] args) {
        Archivio a = new ArchivioFile();
        Servizio s = new Servizio(a);
    }
}
```



`Servizio` conosce solo l'interfaccia (associazione verso `Archivio`); la classe concreta la sceglie e la crea `Main`, che ha le dipendenze «create». Nei test al posto di `ArchivioFile` si passa un oggetto finto. È la Dependency Injection (23.1).

7.7 Una lista di oggetti da avvisare

```
interface Ascoltatore {
    void avvisa(String evento);
}
class Pulsante {
    private final List<Ascoltatore> ascoltatori =
            new ArrayList<>();
    void aggiungi(Ascoltatore a) {
        ascoltatori.add(a);
    }
    void premi() {
        for (Ascoltatore a : ascoltatori)
            a.avvisa("click");
    }
}
class Finestra implements Ascoltatore {
    public void avvisa(String evento) { }
}
```



Una collezione di oggetti di un'**interfaccia**, un metodo per aggiungerli e uno che li percorre chiamando lo stesso metodo su tutti: associazione `*` verso l'interfaccia. È la firma dell'Observer (20.1).

Errori frequenti quando si disegna dal codice

- Scrivere un campo sia fra gli attributi sia come associazione: è un **doppione** (3.9).
- Frecce di generalizzazione **al contrario**: la punta va sempre verso la **superclasse**.
- Realizzazione con la linea continua, o generalizzazione fra classe e interfaccia: `implements` è **tratteggiato**.
- Molteplicità dal lato sbagliato: la molteplicità di una lista sta vicino alla classe degli **elementi**.
- Rombo dal lato della parte: il rombo sta sempre dal lato del **tutto**.
- Disegnare `ArrayList`, `HashMap` o `String` come classi del diagramma.
- Riempire ogni classe di getter e setter: non aggiungono informazioni.

8 Il diagramma dei casi d'uso

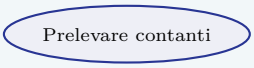
Dispensa: capitolo 6. Chi usa il sistema e per fare che cosa

8.1
Attore
un ruolo, non una persona



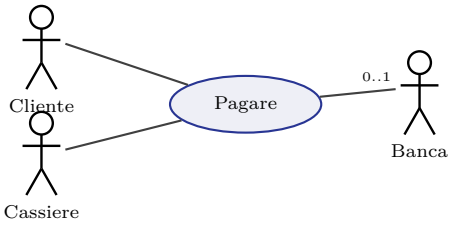
Un'entità **esterna** che interagisce con il sistema: una classe di utenti con lo stesso ruolo, un altro sistema, l'ambiente fisico. Il nome è un **sostantivo**. La stessa persona può avere più ruoli (più attori).

8.2
Caso d'uso
ellisse con un verbo



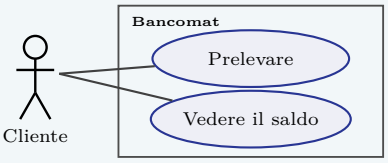
Una funzionalità che dà un **beneficio** a un attore. Il nome contiene un **verbo**. Un caso d'uso è la **classe** di molti scenari con lo stesso obiettivo (come una classe e i suoi oggetti). Premere un pulsante o vedere un errore **non** sono casi d'uso: sono passi.

8.3
Associazione
l'attore esegue il caso d'uso



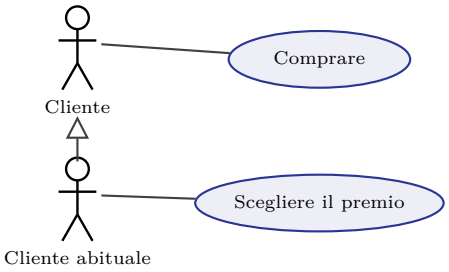
Una linea fra attore e caso d'uso. Più attori sulla **stessa** ellisse **collaborano** in ogni esecuzione (cliente e cassiere insieme), non «uno o l'altro». La cardinalità `0..1` dice che la banca partecipa solo a volte (pagamento con carta).

8.4
Confine del sistema
rettangolo con il nome del sistema oltre le slide



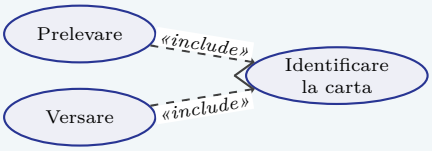
Il rettangolo racchiude i casi d'uso del sistema: gli attori stanno **fuori**. Le slide non lo disegnano, ma è la notazione standard e aiuta a separare ciò che il sistema fa da chi lo usa.

8.5
Generalizzazione tra attori
triangolo vuoto verso l'attore generale



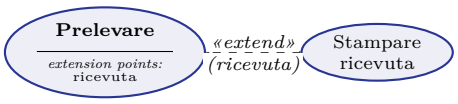
Il cliente abituale **è un** cliente: può fare tutto ciò che fa il cliente (comprare) e in più ha i suoi casi d'uso. È il modo giusto per dire «lo possono fare entrambi». Un attore specializzato senza casi d'uso propri è inutile.

8.6
«include»
dal caso base a quello incluso: sempre



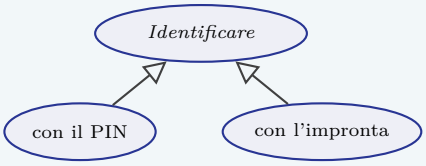
La parte **comune** a più casi d'uso diventa un caso a sé, che i casi base eseguono **ogni volta**. Freccia tratteggiata **dal base all'incluso**: il base dipende dall'incluso (prelevare senza identificare la carta non ha senso). Nel codice: una chiamata che c'è sempre.

8.7
«extend»
dall'estensione al caso base: a volte



Un comportamento **facoltativo** o sotto **condizione**, aggiunto al caso base in un punto preciso (*extension point*, scritto nell'ellisse). Freccia **dall'estensione al base**: il base ha senso anche da solo, l'estensione no. Nel codice: un'aggiunta che il base non conosce.

8.8
Generalizzazione tra casi d'uso
il caso specializzato sostituisce il generale



Un caso d'uso che è un **modo diverso** di fare quello generale, anche molto diverso. Differenza con «extend»: l'estensione **aggiunge** qualcosa al base in un punto preciso, la specializzazione lo **sostituisce**.

Dal testo dell'esercizio al diagramma dei casi d'uso

Nel testo	Nel diagramma
un sostantivo che indica chi usa il sistema (o un sistema esterno)	attore
un verbo che indica un obiettivo dell'utente (*prenotare*, *pagare*)	caso d'uso
«X e Y insieme», «con l'aiuto di», «tramite il servizio di pagamento»	più attori sulla stessa ellisse
«X oppure Y», «anche gli amministratori possono»	generalizzazione tra attori
«ogni volta», «in entrambi i casi bisogna», passi ripetuti in più casi	«include»
«se vuole», «opzionalmente», «può anche», «in caso di»	«extend»
«in due modi», «con il PIN o con l'impronta»	generalizzazione tra casi d'uso
«dopo aver fatto il login», «se è autenticato»	**precondizione** nel testo, non «include»

La specifica di un caso d'uso: il template fully-dressed di Cockburn

Il diagramma non basta: per ogni caso d'uso si scrive una descrizione testuale. Campi del template (in inglese): **Use Case #** e nome, **Goal in Context**, **Preconditions**, **Success End Condition**, **Failed End Condition**, **Primary Actor**, **Trigger**; poi il **Main Scenario** (passi numerati, una colonna per attore e una per il sistema), le **Extension** (ognuna parte da un passo, con una condizione, e termina o torna al main scenario) e gli **Open Issues** (vuoti alla consegna). Formati più leggeri: **brief** (un paragrafo) e **casual** (più paragrafi), per le prime fasi.

Errori tipici

- Casi d'uso che non sono obiettivi dell'utente: «premere OK», «vedere l'errore del PIN» sono **passi**.
- Due attori sulla stessa ellisse per dire «lo può fare l'uno o l'altro»: serve la **generalizzazione**.
- «include» per dire «prima»: il login è una **precondizione**, non un caso incluso.
- Frecce al contrario: «include» va dal base all'incluso, «extend» dall'estensione al base.
- Diagrammi pieni di frecce incrociate: i dettagli vanno nella descrizione testuale.

9 Il diagramma di sequenza

Dispensa: capitolo 8. Chi chiama chi, nel tempo, in uno scenario

9.1 Partecipante e linea di vita

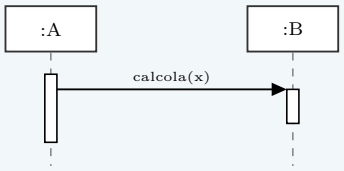
il tempo scorre verso il basso



Ogni partecipante è un **oggetto**: `nome : Classe` sottolineato (anche senza nome, `:Cliente`), oppure un nome che fa capire la classe (`unProdotto`). La linea tratteggiata è la sua **vita**. Gli attori, con l'omino, stanno a sinistra.

9.2 Messaggio sincrono e attivazione

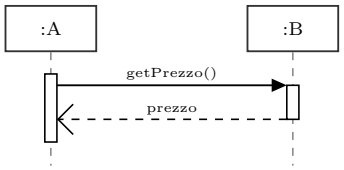
freccia a triangolo pieno: una chiamata di metodo



`b.calcola(x)` nel codice di `A`. Chi chiama **aspetta** la fine del metodo. Il rettangolino sulla linea di vita (**attivazione**) è il tempo in cui il metodo è in esecuzione. Ogni messaggio che `:B` riceve dev'essere un'**operazione** della classe `B`.

9.3 Ritorno

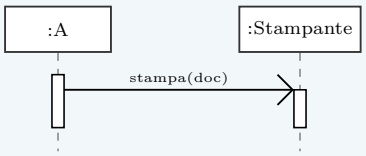
tratteggiato, si può omettere



`double prezzo = b.getPrezzo();`: la risposta torna indietro con una freccia tratteggiata, etichettata con il valore. Si disegna quando il valore serve a capire il seguito.

9.4 Messaggio asincrono

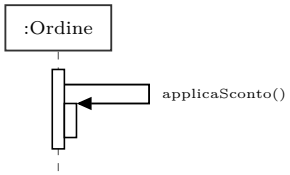
freccia aperta: chi invia non aspetta



Il mittente prosegue subito, senza aspettare: un messaggio in una coda, un evento, un compito avviato su un altro *thread* (`CompletableFuture.runAsync`, `new Thread(…).start()`).

9.5 Chiamata a se stesso

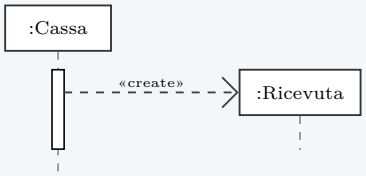
il messaggio torna sulla stessa linea



`this.applicaSconto()`, o `applicaSconto()` senza oggetto davanti: l'oggetto chiama un proprio metodo. Sulla barra di attivazione se ne appoggia una seconda, un po' spostata.

9.6 Creazione

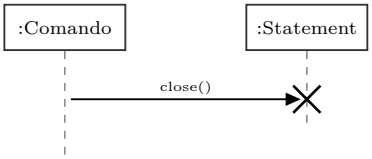
il rettangolo alla punta della freccia



`new Ricevuta(totale)`: l'oggetto nasce in quel momento, quindi il suo rettangolo sta **più in basso**, alla punta del messaggio (`new` o «create»). Gli oggetti che esistono già (anche quelli `static`) partono dalla cima.

9.7 Distruzione

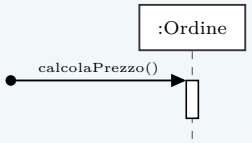
una croce chiude la linea di vita



L'oggetto finisce di esistere: `close()`, `dispose()`, o non serve più. Può distruggerlo un altro oggetto o chiudersi da solo (croce alla fine della sua linea).

9.8 Messaggio trovato

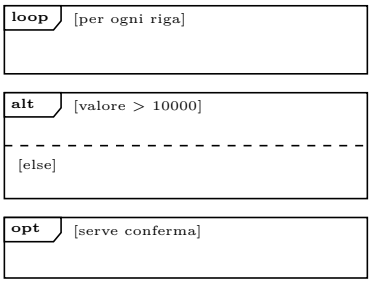
un pallino: arriva da fuori



La chiamata arriva da qualcuno che il diagramma non mostra: è il punto di partenza del metodo che si sta descrivendo. In alternativa si disegna l'attore che avvia lo scenario.

9.9 Frame loop, alt, opt

cicli e condizioni (UML 2)



Riquadri che racchiudono i messaggi ripetuti o condizionati, con la condizione fra quadre: **loop** = `for`/`while`; **alt** = `if-else` (i rami separati dal tratteggio, l'ultimo `[else]`); **opt** = `if` senza `else`. Esistono anche **par** (in parallelo) e **critical** (sezione critica).

Dal codice Java al diagramma di sequenza

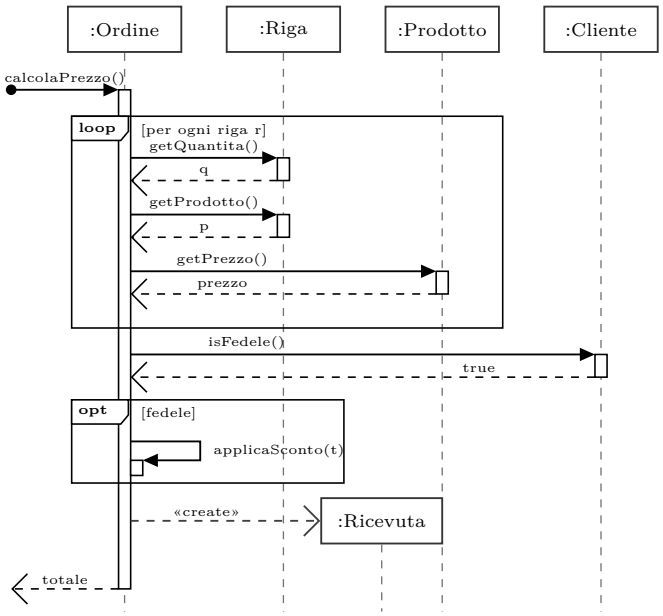
Nel codice del metodo	Nel diagramma
`b.metodo(x)`	messaggio sincrono `metodo(x)` verso la linea di `b`
`y = b.metodo()`	messaggio sincrono e ritorno tratteggiato `y`
`this.metodo()` o `metodo()`	chiamata a se stesso
`new B(…)`	creazione: il rettangolo di `:B` alla punta della freccia
`B.metodoStatico()`	messaggio verso il partecipante `B` (la classe)
`for`, `while`, `forEach`	frame **loop** con la condizione
`if … else …`	frame **alt** con due zone
`if` senza `else`	frame **opt**
`CompletableFuture.runAsync`, eventi, code	messaggio asincrono
`close()`, `dispose()`, oggetto non più usato	croce di distruzione

Si disegna **un** scenario alla volta, con gli oggetti **davvero** coinvolti; un diagramma pieno di frame annidati vuol dire che serve un diagramma di attività (**12.1**).

9.10 Un metodo e il suo diagramma di sequenza

```java
class Ordine {
    private final List<Riga> righe =
            new ArrayList<>();
    private Cliente cliente;

    double calcolaPrezzo() {
        double totale = 0;
        for (Riga r : righe) {            // loop
            int q = r.getQuantita();
            Prodotto p = r.getProdotto();
            totale += q * p.getPrezzo();
        }
        if (cliente.isFedele())           // opt
            totale = applicaSconto(totale);
        new Ricevuta(totale);        // creazione
        return totale;
    }
    double applicaSconto(double t) {
        return t * 0.9;
    }
}
class Riga {
    private int quantita;
    private Prodotto prodotto;
    int getQuantita() { return quantita; }
    Prodotto getProdotto() { return prodotto; }
}
class Prodotto {
    double getPrezzo() { return 10; }
}
class Cliente {
    boolean isFedele() { return true; }
}
class Ricevuta {
    Ricevuta(double totale) { }
}
```

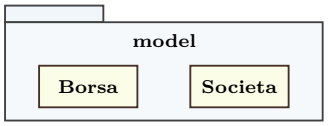


Una linea di vita per ogni oggetto che il metodo usa; un messaggio per ogni chiamata, nell'ordine del codice. Il `for` diventa **loop**, l'`if` senza `else` **opt**, la chiamata a `applicaSconto` un **autoanello**, il `new` una **creazione** più in basso. Le chiamate `getQuantita()`, `getProdotto()`… sono operazioni delle classi `Riga`, `Prodotto`: i due diagrammi devono essere coerenti.

10 Package, componenti e deployment

Dispensa: capitoli 8 e 9. Come è diviso il sistema e dove gira

10.1
Package
una cartella con la linguetta



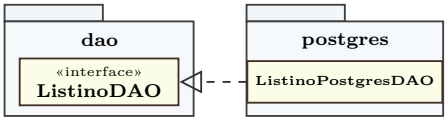
Raggruppa classi (e altri package). In Java è un `package`: le classi della cartella `model` iniziano con `package model;`. Dentro si può disegnare il diagramma delle classi del package, o lasciarlo vuoto.

10.2
Dipendenza fra package
freccia tratteggiata: gli import



`gui` dipende da `controller`, che dipende da `model`: nel codice sono gli `import` (`import model.Borsa;` in una classe del controller). Il `model` non importa nulla: si può cambiare l'interfaccia grafica senza toccarlo. Un `import` contrario alle frecce viola l'architettura.

10.3
Interfacce fra package
realizzazione e dipendenza attraverso un'interfaccia



Il controller usa l'interfaccia `ListinoDAO` (package `dao`); la classe per PostgreSQL la realizza in un altro package. Per cambiare database si scrive un'altra implementazione, e chi usa `ListinoDAO` non cambia.

10.4
Componente
«component» e l'icona in alto a destra



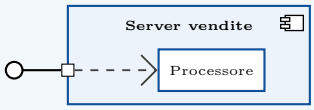
Un pezzo del sistema che si costruisce e si sostituisce da solo: un eseguibile, una libreria (`.jar`), un servizio, un database. Contiene classi e package. Il diagramma dei componenti dice **che cosa** c'è e come si collega, **non dove** gira.

10.5
Interfacce fornite e richieste
pallina, presa e il loro incastro



`Contabilità` **fornisce** l'interfaccia `Registro` (pallina: la implementa); `Vendite` la **richiede** (presa: ne ha bisogno). L'incastro è il collegamento. In Java: una `interface`, una classe che la implementa e una che la riceve nel costruttore (**10.10**).

10.6
Porta
quadratino sul bordo



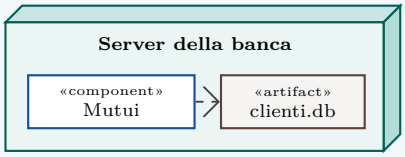
Il punto in cui i dati entrano o escono da un componente (con eventuali trasformazioni o serializzazioni). Un componente che ne contiene altri è un **sottosistema** («subsystem»): la porta inoltra all'interno ciò che arriva dall'esterno.

10.7
Nodo
parallelepipedo: una macchina o un dispositivo



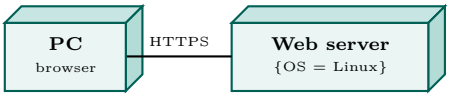
Un elemento **fisico**: un server, un PC, un telefono, un sensore. Il nome è un'istanza, `nome : Tipo`. Un nodo può contenere **ambienti di esecuzione** (un DBMS, un container EJB, Docker), anche loro disegnati come nodi.

10.8
Artefatto e componenti nel nodo
«artifact»: un file prodotto o usato



Dentro un nodo si disegnano i **componenti** che vi girano e gli **artefatti**: file eseguibili (`.jar`, `.war`), database, file di configurazione. Così il diagramma di deployment si collega a quello dei componenti.

10.9
Connessione e valori etichettati
una linea con il protocollo; {nome = valore}



Le connessioni sono collegamenti **fisici**, etichettati con il protocollo (TCP/IP, HTTP, JDBC, Java RMI). I **valori etichettati** `{nome = valore}` aggiungono dettagli: il sistema operativo, il software, quanti nodi uguali ci sono (`{number deployed = 3}`).

10.10 Interfaccia fornita e richiesta, nel codice

```java
interface Registro {          // l'interfaccia
    void registra(String vendita);
}
class Contabilita implements Registro {  // fornisce
    public void registra(String vendita) { }
}
class Vendite {
    private final Registro registro;     // richiede
    Vendite(Registro r) { registro = r; }
    void vendi(String articolo) {
        registro.registra(articolo);
    }
}
public class Assemblaggio {
    public static void main(String[] args) {
        Vendite v = new Vendite(new Contabilita());
    }
}
```



La **pallina** è la classe che `implements` l'interfaccia; la **presa** è la classe che la riceve (nel costruttore) e la usa. L'incastro avviene dove si assemblano i pezzi (`main`): è l'unico punto che conosce tutti e due. È la stessa idea della Dependency Injection (7.6).

Package, componenti o deployment?

- **Package**: come è **organizzato il codice** (cartelle, `import`). Si sceglie con l'architettura, poi si disegna un diagramma delle classi per package.
- **Componenti**: **che cosa** è stato costruito e chi offre quale interfaccia a chi, in modo logico.
- **Deployment**: **dove** gira ogni pezzo, su quale macchina, con quale collegamento fisico. Lo stesso componente può comparire nel diagramma dei componenti e, dentro un nodo, in quello di deployment.

11 Il diagramma di stato

Dispensa: capitolo 9. Come reagisce un oggetto agli eventi, a seconda dello stato in cui si trova

11.1
Stato
rettangolo con gli angoli arrotondati



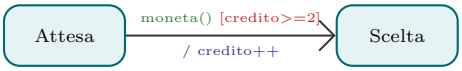
Una situazione **stabile** in cui l'oggetto reagisce agli eventi in un certo modo; ci resta finché un evento non lo fa cambiare. In ogni momento è attivo **un solo** stato (uno per regione, **11.7**). Nel codice: il valore di una variabile di stato, o l'oggetto stato corrente del pattern State.

11.2
Stato iniziale e finale
pallino pieno; pallino cerchiato



Due **pseudostati**: da quello iniziale parte la prima transizione (non ha frecce entranti); in quello finale l'oggetto ha finito (non ha frecce uscenti). La prima transizione può avere un'azione (`/ credito = 0`): il costruttore.

11.3
Transizione
`evento [guardia] / azioni`



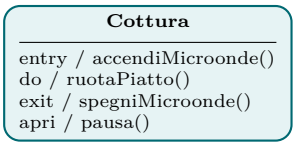
Scatta quando arriva l'**evento** (una chiamata di metodo) **e** la **guardia** è vera; poi esegue le **azioni**. Tutte e tre le parti sono facoltative: senza evento né guardia la transizione è spontanea. Se più transizioni sono attivabili ne scatta una a caso: le guardie dello stesso evento devono **escludersi**.

11.4
Autotransizione
esce dallo stato e ci rientra



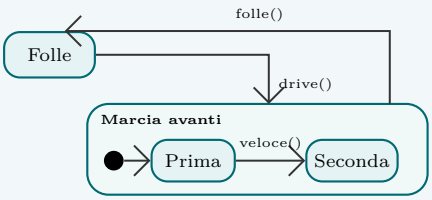
L'evento cambia i dati ma non lo stato. Attenzione: l'autotransizione **esce** e **rientra**, quindi esegue le attività `exit` ed `entry` (**11.5**); una reazione **interna**, scritta dentro lo stato, no.

11.5
Attività interne
entry, do, exit



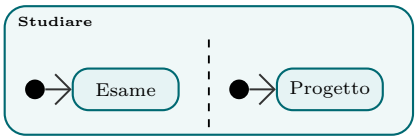
`entry /` all'ingresso, `do /` finché si resta nello stato, `exit /` all'uscita (per qualunque transizione). Le altre righe `evento [guardia] / azione` sono **reazioni interne**: non fanno uscire dallo stato.

11.6
Stato composito
uno stato che ne contiene altri



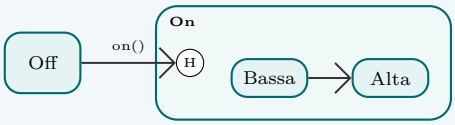
Dentro c'è un intero diagramma di stato. Una transizione che parte dal **bordo** vale per **tutti** i sottostati (`folle()` da qualunque marcia); una che arriva sul bordo entra dallo stato iniziale interno (`drive()` porta in `Prima`).

11.7
Regioni parallele
separate dal tratteggio



Comportamenti che procedono **in parallelo**: in ogni regione è attivo uno stato (si prepara l'esame **e** si fa il progetto). Quando si esce dallo stato composito, tutte le regioni terminano.

11.8
History
H: ultimo sottostato; H*: a ogni livello

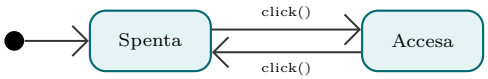


Entrando dalla **H** si torna nel sottostato in cui ci si trovava quando si è usciti (la lampada si riaccende all'ultimo livello di luce); la prima volta si parte dallo stato iniziale. **H** (*shallow*) ricorda solo il primo livello; **H*** (*deep*) l'intera configurazione, sottostati dei sottostati compresi.

11.9 Una variabile di stato e uno switch

```
class Lampada {
    enum Stato { SPENTA, ACCESA }
    private Stato stato = Stato.SPENTA;

    void click() {
        switch (stato) {
            case SPENTA -> stato = Stato.ACCESA;
            case ACCESA -> stato = Stato.SPENTA;
        }
    }
}
```

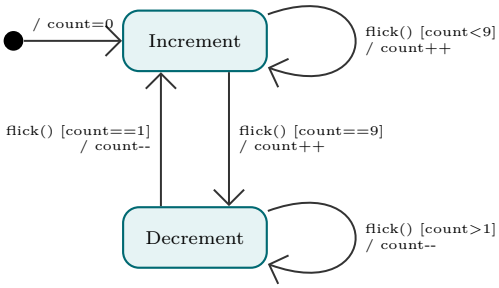


La variabile che dice «in che fase sono» (spesso un'`enum`) è lo **stato**: ogni suo valore è uno stato del diagramma, il valore iniziale è la prima transizione. Ogni metodo che la cambia è un **evento**. Lo stesso evento (`click()`) fa cose diverse a seconda dello stato: è ciò che il diagramma mostra.

11.10 Guardie e azioni: quali variabili diventano stati

```
public class Counter {
    private int count = 0;
    private String mode = "increment";

    public void flick() {
        if (mode.equals("increment"))
            count++;
        else
            count--;
        if (count == 10)
            mode = "decrement";
        else if (count == 0)
            mode = "increment";
    }
}
```



`mode` decide il comportamento: i suoi valori sono gli **stati**. `count` no (sarebbero 11 stati): compare nelle **guardie** e nelle **azioni**. Le guardie guardano il valore **prima** dell'azione: con `count==9` lo `flick()` porta a 10 e fa passare in Decrement.

Dal diagramma al codice: switch annidati o pattern State

Due modi. Con gli **switch annidati** il primo sceglie lo stato corrente, il secondo l'evento arrivato: funziona, ma con molti stati diventa enorme. Con il **pattern State** (21.1) ogni stato è una classe:

Nel diagramma	Nel codice con il pattern State
stato	una sottoclasse di `State` (`Attesa`, …)
evento (trigger)	un metodo di `State`, vuoto nella classe base
guardia	un `if` dentro il metodo
azione	il corpo del metodo
transizione	la chiamata `contesto.setState(nuovoStato)`
evento ignorato in uno stato	metodo non ridefinito: resta quello vuoto

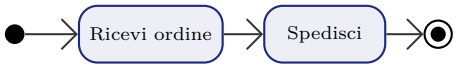
Errori tipici

- Due transizioni con lo stesso evento e guardie che si sovrappongono: comportamento **non deterministico**.
- Un'autotransizione al posto di una reazione interna: rieseguire `entry` può azzerare i dati.
- Uno stato per ogni valore di un contatore: i numeri vanno in guardie e azioni, non negli stati.
- Confondere stato e attività: lo stato si **è**, l'attività si **fa** (12.1).

12 Il diagramma di attività

Dispensa: capitolo 9. Un flusso di lavoro o un algoritmo, anche concorrente

12.1
Azione, inizio e fine
rettangoli arrotondati collegati da frecce



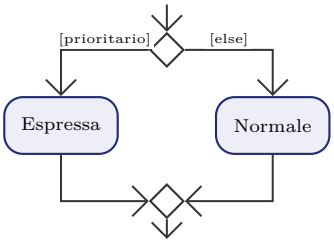
Un'**azione** è un passo di elaborazione (un verbo); le frecce sono il **flusso**: si entra quando l'azione inizia, si esce quando finisce. Il nodo finale chiude **l'intera** attività, anche gli altri flussi ancora in corso.

12.2
Fine di un flusso
cerchio con la croce
oltre le slide



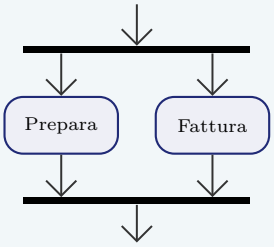
Termina **solo quel flusso**, non tutta l'attività: utile dopo un fork, quando un ramo finisce prima degli altri senza doverli interrompere.

12.3
Branch e merge
il rombo: un solo ramo



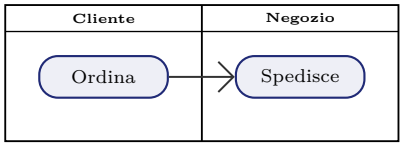
Il **branch** è un `if`: un arco entra, più archi escono, ognuno con una **guardia** fra quadre (mutuamente esclusive, `[else]` per il resto). Il **merge** è la fine dell'`if`: più archi entrano, ne esce uno. Un arco che torna indietro a un rombo è un **ciclo**.

12.4
Fork e join
la barra: tutti i rami, insieme



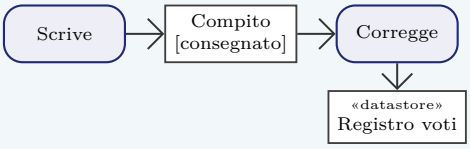
Il **fork** avvia più flussi **concorrenti** (thread): dopo la barra si percorrono **tutti** i rami. Il **join** aspetta che arrivino **tutti** prima di proseguire. Il rombo sceglie, la barra sdoppia.

12.5
Swimlane
corsie: chi fa che cosa



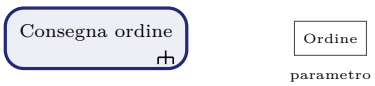
Il diagramma si divide in **corsie**, una per classe, ruolo, reparto o sistema; ogni azione sta nella corsia di chi la esegue. Così il diagramma di attività mostra anche la collaborazione fra oggetti diversi.

12.6
Nodo oggetto e datastore
l'oggetto che passa da un'azione all'altra



Un rettangolo con il nome di un oggetto e, fra quadre, il suo **stato** in quel punto del flusso. Il **«datastore»** è un archivio persistente: i dati restano anche quando l'attività finisce.

12.7
Macro-attività
il rastrello: c'è un sotto-diagramma



Un compito complesso diventa una sola azione, con il simbolo del **rastrello**, e si dettaglia in un altro diagramma. Il dettaglio è un'attività con un nome e dei **parametri**: rettangoli sul bordo per i dati in ingresso e in uscita.

12.8
Segnali
tempo, ricevuto, inviato

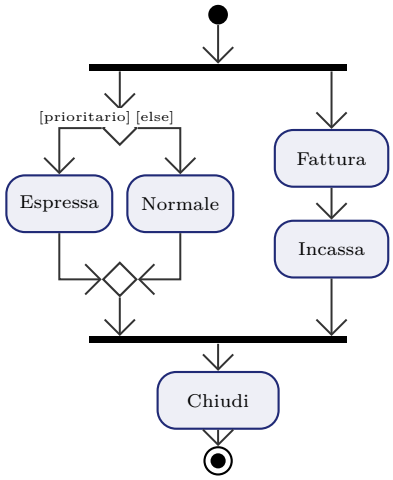


Eventi **esterni** e **asincroni**: la **clessidra** è un segnale di tempo («due ore prima del volo», «dopo 48 ore»); il pentagono con la rientranza è un segnale **ricevuto** (accept, «arriva il taxi»); quello a punta un segnale **inviato** (send).

12.9 Dal codice al diagramma di attività

```java
import java.util.concurrent.CompletableFuture;

class GestioneOrdini {
    void gestisci(Ordine o) {
        // fork: i due flussi partono insieme
        CompletableFuture<Void> spedizione =
            CompletableFuture.runAsync(() -> {
                if (o.prioritario()) espressa(o);
                else normale(o);
            });
        CompletableFuture<Void> pagamento =
            CompletableFuture.runAsync(() -> {
                fattura(o);
                incassa(o);
            });
        // join: aspetta che finiscano entrambi
        CompletableFuture
            .allOf(spedizione, pagamento).join();
        chiudi(o);
    }
    void espressa(Ordine o) { }
    void normale(Ordine o) { }
    void fattura(Ordine o) { }
    void incassa(Ordine o) { }
    void chiudi(Ordine o) { }
}
class Ordine {
    boolean prioritario() { return false; }
}
```



Un **if**/`else` diventa branch e merge (rombi); due compiti avviati **in parallelo** diventano un fork, e l'attesa che finiscano entrambi (`allOf(…).join()`) un join (barre). Un `while` è un rombo con un arco che torna indietro.

Errori tipici

- Rombo al posto della barra (o il contrario): dopo il **rombo** si prende **un** ramo, dopo la **barra tutti**; il merge lascia passare il primo flusso, il join li aspetta tutti.
- Guardie che non si escludono, o che non coprono tutti i casi (manca `[else]`).
- Un nodo finale nel ramo di un fork: chiude anche gli altri rami (se non è voluto, serve la fine di flusso).
- Un flusso che si ferma nel nulla: ogni percorso deve arrivare a un nodo finale.

13 Quale diagramma?

Dispensa: capitoli 6--9. Dalla domanda dell'esercizio al diagramma giusto

Che cosa chiede l'esercizio

La domanda	Il diagramma	Argomento
Chi usa il sistema e che cosa può farci? Quali sono i requisiti funzionali?	casi d'uso	8.1
Quali concetti esistono nel dominio, con quali dati e legami?	classi (dominio)	1.1
Come sono fatte le classi Java del progetto? Disegna il diagramma di questo codice.	classi (progetto)	3.1
Chi chiama chi, e in che ordine, in uno scenario? Che cosa fa questo metodo?	sequenza	9.1
Come reagisce un oggetto agli eventi? Ha delle fasi? Che cosa fa un'interfaccia grafica?	stato	11.1
Qual è il flusso di lavoro, chi fa che cosa, che cosa avviene in parallelo?	attività	12.1
Come si divide il codice in moduli? Quali dipendenze ci sono fra i moduli?	package	10.1
Quali pezzi compongono il sistema e quali interfacce offrono?	componenti	10.4
Su quali macchine gira il sistema, e come sono collegate?	deployment	10.7

I diagrammi di UML 2: due famiglie

- **Statici** (strutturali): **com'è fatto** il sistema. Classi, package, componenti, deployment (e oggetti).
- **Dinamici** (comportamentali): **che cosa fa** nel tempo. Casi d'uso, stato, attività e quelli di **interazione**: sequenza (il più usato), comunicazione, interaction overview, timing.

UML 2 ha 13 tipi di diagrammi; i più usati sono quello delle classi e quello di sequenza. Vale sempre la regola delle **informazioni soppresse**: ciò che un diagramma non mostra non è per forza assente.

Quelli che si confondono

Stato o attività? Lo **stato** descrive **un oggetto** che aspetta eventi e cambia comportamento (rettangoli = situazioni in cui l'oggetto *si trova*). L'**attività** descrive un **processo** che avanza da solo (rettangoli = azioni che *si fanno*), anche fra più oggetti (swimlane).

Sequenza o attività? La **sequenza** mostra **chi parla con chi** in un solo scenario; l'**attività** mostra la **logica** (decisioni, cicli, parallelismo). Per un algoritmo, meglio l'attività.

Dominio o progetto? Nel modello di **dominio** (concettuale): niente metodi (al più responsabilità), niente tipi e visibilità, niente verso di navigazione, niente «Sistema» né «id». Nel **progetto**: tutto questo c'è, e il diagramma corrisponde al codice.

Classi o oggetti? Il diagramma delle **classi** vale per tutte le istanze (molteplicità); un diagramma di **oggetti** oltre le slide fotografa un momento, con i valori dei campi.

Componenti o deployment? I **componenti** dicono che cosa c'è e chi offre quale interfaccia; il **deployment** dice su quale macchina gira.

Le regole che valgono in tutti i diagrammi

- Nomi delle classi al **singolare**; attori sostantivi, casi d'uso e azioni con un **verbo**.
- Le frecce **tratteggiate** con la punta aperta sono dipendenze («include», «extend», «call», «create», «use», le dipendenze fra package e fra componenti).
- Un diagramma serve a **comunicare**: se è troppo affollato va diviso o semplificato (prevalgono le regole descrittive).
- I diagrammi dello stesso sistema devono essere **coerenti**: ogni messaggio di un diagramma di sequenza è un'operazione nel diagramma delle classi, ogni evento di un diagramma di stato è un metodo.

Parte II

Riconoscere i design pattern

14 Come si riconosce un pattern

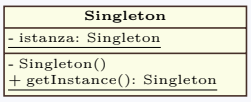
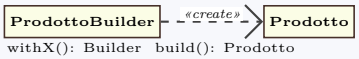
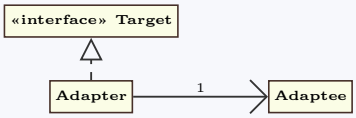
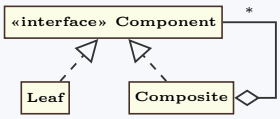
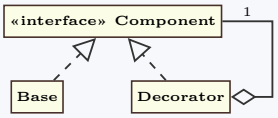
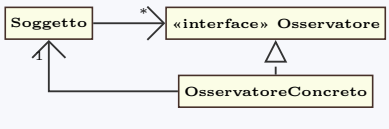
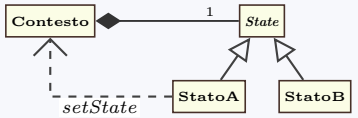
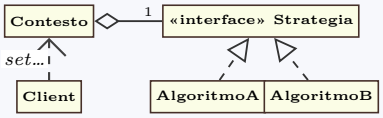
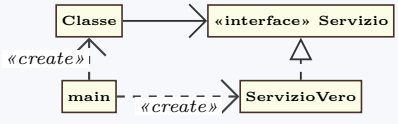
Dispensa: capitolo 15. Dal testo, dal codice o dal diagramma al nome del pattern

La procedura, in cinque domande

1. **C'è un'interfaccia (o una classe astratta) al centro?** Quasi tutti i pattern ne hanno una; non ce l'hanno il Singleton e il Builder senza Director.
2. **Chi la implementa ha anche un campo dello stesso tipo?** Uno solo → Decorator; una collezione → Composite. Se invece ha un campo di un tipo **diverso**, con un'altra interfaccia, e ne traduce le chiamate → Adapter.
3. **Chi crea gli oggetti?** Costruttore privato e metodo statico che restituisce sempre lo stesso oggetto → Singleton; metodi che restituiscono `this` e un `build()` finale → Builder; dipendenze ricevute nel costruttore invece di `new` → Dependency Injection.
4. **Chi cambia l'oggetto a cui un campo punta?** Gli oggetti puntati stessi (`setState`) → State; il client (`setStrategia`) → Strategy.
5. **C'è una lista che riceve iscrizioni e avvisa tutti?** `attach`, `detach` e un ciclo che chiama `update()` → Observer.

Se due pattern hanno la stessa forma (State e Strategy, Decorator e Proxy…) decide l'**intento**: che problema risolve (**24.1**).

Le firme: il disegno tipico e che cosa cercare nel codice

Pattern	Disegno tipico	Nel codice
Singleton 15.1		costruttore `private`; campo `private static` del tipo della classe stessa; `public static getInstance()` con `if (istanza == null) istanza = new …`; chi lo usa non fa mai `new`
Builder 16.1		metodi `withX(…)` (o `setX`) che fanno `return this`; `build()` che controlla e fa `new Prodotto(…)`; uso a catena `new XBuilder(…).withA().withB().build()`; con il Director, un'interfaccia di passi e più builder
Adapter 17.1		`class Adapter implements Target` con un campo `Adaptee` di un tipo **diverso** (o `extends Adaptee`); ogni metodo converte i parametri e chiama un metodo con un altro nome: `evaluate(…)` chiama `calcolaIMC(…)`
Composite 18.1		`class Composite implements Component` con `List<Component> figli`, `add` e `remove`; il metodo fa `for (Component f : figli) f.metodo();` (ricorsione su un albero); anche le foglie implementano `Component`
Decorator 19.1		`class Decorator implements Component` con **un** campo `Component` ricevuto nel costruttore; il metodo fa `avvolto.metodo()` più qualcosa (`+ 1.50`); uso `new A(new B(new Base()))`
Observer 20.1		`List<Osservatore>`; `attach`/`detach` (o `addXListener`); un metodo di notifica con `for (Osservatore o : osservatori) o.update();`; il setter dello stato chiama la notifica; l'osservatore rilegge lo stato dal soggetto
State 21.1		campo `State stato`; ogni metodo del contesto fa `stato.evento(this)`; una sottoclasse per stato; gli **stati** chiamano `contesto.setState(…)`; stati spesso in campi `static`
Strategy 22.1		campo di un'interfaccia con un solo metodo (l'algoritmo), impostato **dal client** con il costruttore o un setter; il contesto fa `strategia.calcola(…)`; le strategie non si conoscono; spesso lambda o `Comparator`
Dependency Injection 23.1		costruttore che riceve un'**interfaccia** e la salva in un campo `final`; nella classe nessun `new` della classe concreta; il `main` o il test fanno `new Classe(new Dipendenza())`; `@Autowired`, `@Inject`

Le parole del testo

Se il testo dice...	pensa a
«una sola istanza», «unico», «condiviso da tutta l'applicazione», «accesso globale», «la connessione al database»	**Singleton**
«molti parametri, alcuni facoltativi», «costruito passo per passo», «configurazioni diverse», «stessa procedura, risultati diversi»	**Builder**
«riusare una classe, una libreria o un servizio esistente», «interfaccia incompatibile», «non si può modificare», «convertire unità o formati»	**Adapter**
«gerarchia», «albero», «contiene elementi che contengono altri elementi», «cartelle e file», «menu e sottomenu», «trattare in modo uniforme singoli e gruppi»	**Composite**
«aggiungere funzionalità a un oggetto», «a tempo di esecuzione», «combinazioni, anche ripetute», «senza una sottoclasse per ogni combinazione», «ingredienti, optional, filtri»	**Decorator**
«quando X cambia, aggiornare automaticamente Y», «notificare», «iscriversi», «abbonati», «più viste degli stessi dati», «eventi»	**Observer**
«si comporta diversamente a seconda dello stato», «fasi», «transizioni», «distributore automatico», un diagramma di stato da implementare	**State**
«modi diversi di calcolare», «politiche», «l'utente sceglie il metodo», «cambiare l'algoritmo a tempo di esecuzione», «aggiungerne di nuovi senza modificare la classe»	**Strategy**
«testare senza il servizio reale», «oggetto fittizio», «cambiare implementazione senza modificare la classe», «configurare dall'esterno»	**Dependency Injection**

Le famiglie della GoF

I 23 pattern della *Gang of Four* (Gamma, Helm, Johnson, Vlissides, 1995) sono divisi secondo il problema che risolvono. In grassetto quelli che il corso approfondisce; gli altri sono solo nominati nelle slide (24.2).

- **Creazionali**, come si **creano** gli oggetti: **Singleton**, **Builder**, Abstract Factory, Factory Method, Prototype.
- **Strutturali**, come si **compongono** classi e oggetti: **Adapter**, **Composite**, **Decorator**, Bridge, Facade, Proxy, Flyweight.
- **Comportamentali**, come si dividono **algoritmi e responsabilità**: **Observer**, **State**, **Strategy**, Template Method, Visitor (e altri che le slide non riportano, come Iterator e Command).
- Fuori dal catalogo, **moderno**: la **Dependency Injection**.

Ogni pattern ha quattro parti: **nome**, **problema**, **soluzione** (uno schema di classi da adattare) e **conseguenze**. Prima si definisce il problema, poi si cerca il pattern: applicato dove non serve, aggiunge solo classi.

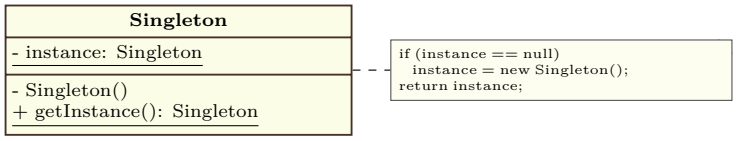
15 Singleton

Dispensa: capitolo 15. Pattern creazionale della GoF

Carta d'identità

Intento	Garantire che una classe abbia **una e una sola istanza**, e dare a tutti un **punto di accesso globale** a quell'istanza.
Nel testo	«un solo», «unico», «una e una sola istanza», «condiviso da tutta l'applicazione», «accessibile da ovunque»; gli esempi tipici: la **connessione al database**, la **configurazione**, il registro dei **log**, la coda di stampa.
Nel codice	costruttore `private`; un campo `private static` del **tipo della classe stessa**; un metodo `public static getInstance()` che crea l'oggetto solo se il campo è `null` e poi restituisce sempre quello; spesso `clone()` ridefinito per lanciare un'eccezione. Chi lo usa scrive `Classe.getInstance()`, mai `new`.
Nel diagramma	una classe sola, con un attributo sottolineato del suo stesso tipo, il costruttore con il `-` e un'operazione sottolineata che restituisce la classe.
Varianti	istanza creata subito, quando la classe viene caricata (`private static final Singleton ISTANZA = new Singleton();`): niente inizializzazione pigra, ma sicura con i thread; `getInstance` con `synchronized`; un numero **fisso** di istanze (al massimo 3, in un array: 15.3); un'`enum` con un solo valore oltre le slide.
Da non confondere	una classe con soli metodi `static` (come `Math`): non c'è un oggetto, non si può personalizzare la creazione né passarlo come parametro. E la Dependency Injection (23.1): lì l'oggetto viene **passato** a chi lo usa, qui chi lo usa **va a prenderlo**.

15.1 La struttura



Un solo partecipante: la classe stessa. Il campo statico appartiene alla classe e sopravvive mentre gli oggetti nascono e muoiono; il costruttore privato lo può chiamare solo `getInstance`, che lo chiama una volta sola (*lazy initialization*). Se un'altra classe prova `new Singleton()`, la ferma il compilatore: `Singleton() has private access`.

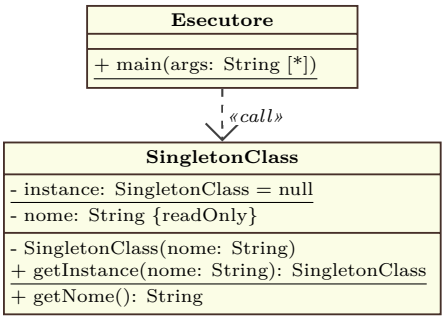
15.2 L'esempio del corso: un nome che non cambia

Esecutore.java

```java
public class Esecutore {
    public static void main(String[] args) {
        SingletonClass sc1 =
                SingletonClass.getInstance("uno");
        SingletonClass sc2 =
                SingletonClass.getInstance("due");
        System.out.println(sc1.getNome()
                + " --- " + sc2.getNome());
        System.out.println("stesso oggetto? "
                + (sc1 == sc2));
    }
}

class SingletonClass {
    private static SingletonClass instance = null;
    private final String nome;

    private SingletonClass(String nome) {
        this.nome = nome;
    }
    public static SingletonClass getInstance(
            String nome) {
        if (instance == null)
            instance = new SingletonClass(nome);
        return instance;
    }
    public String getNome() { return nome; }
}
```



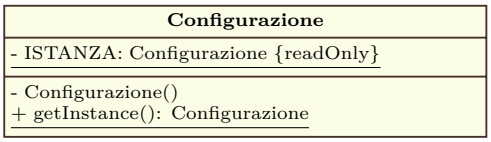
Terminale

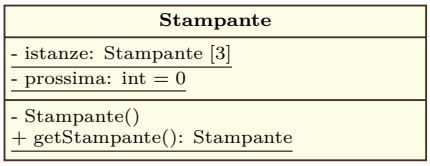
```
$ javac Esecutore.java
$ java Esecutore
uno --- uno
stesso oggetto? true
```

La seconda chiamata trova `instance` già impostato: non chiama il costruttore e il parametro `"due"` viene **ignorato**. Per questo un `getInstance` con parametri è ingannevole: meglio configurare l'oggetto una volta, all'avvio. (Nel codice del corso `nome` è un campo `static`; qui è un campo dell'oggetto, che è la scelta più comune.)

15.3 Le varianti nel codice

```java
// creato subito, quando la classe viene caricata
class Configurazione {
    private static final Configurazione ISTANZA =
            new Configurazione();
    private Configurazione() { }
    public static Configurazione getInstance() {
        return ISTANZA;
    }
}
// pigro ma sicuro con i thread
class Registro {
    private static Registro istanza;
    private Registro() { }
    public static synchronized Registro
            getInstance() {
        if (istanza == null)
            istanza = new Registro();
        return istanza;
    }
}
// al massimo tre istanze, assegnate a turno
class Stampante {
    private static final Stampante[] istanze =
            new Stampante[3];
    private static int prossima = 0;
    private Stampante() { }
    public static Stampante getStampante() {
        int i = prossima;
        prossima = (prossima + 1) % 3;
        if (istanze[i] == null)
            istanze[i] = new Stampante();
        return istanze[i];
    }
}
```





Il diagramma non cambia: costruttore privato, campo e metodo statici. Il campo creato subito è `final`, quindi `{readOnly}`. Con un numero fisso di istanze il campo è un array: la molteplicità `[3]`.

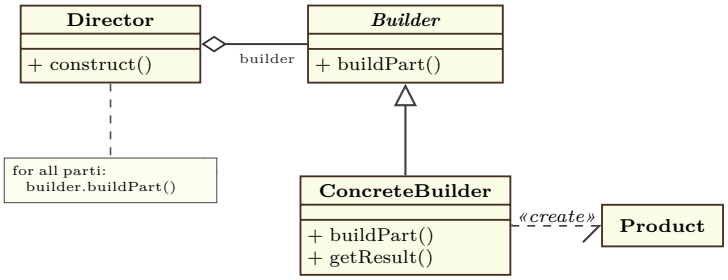
16 Builder

Dispensa: capitolo 15. Pattern creazionale della GoF

Carta d'identità

Intento	Rendere la costruzione di un oggetto complesso **flessibile e scomponibile in passi**, separandola dal costruttore dell'oggetto; con un Director, gli **stessi passi** producono **rappresentazioni diverse**.
Nel testo	«molti parametri, alcuni obbligatori e molti facoltativi», «configurazioni diverse dello stesso prodotto», «costruito un pezzo alla volta», «in futuro le opzioni aumenteranno», «controllare i dati prima di creare l'oggetto», «la stessa procedura deve produrre risultati diversi» (una casa e un grattacielo, una scheda tecnica e un preventivo).
Nel codice	una classe `XBuilder` con gli stessi campi del prodotto; i parametri obbligatori nel suo costruttore; metodi `withY(…)` (o `setY`) che impostano un'opzione e fanno `return this`, così le chiamate si concatenano; un metodo finale `build()` (o `getResult()`) che controlla i dati e fa `new Prodotto(…)`. Il prodotto ha campi `final` e un costruttore non pubblico.
Nel diagramma	il Builder con una dipendenza «create» verso il prodotto, e operazioni che restituiscono il Builder stesso. Con il Director: un'interfaccia (o classe astratta) Builder con più ConcreteBuilder, e il Director che la usa.
Varianti	senza Director (il client chiama i passi: **16.2**) o con il Director che conosce le «ricette» (**16.3**); il builder come classe annidata statica del prodotto (`Computer.builder()`) oltre le slide. Nella libreria di Java: `StringBuilder`, `Locale.Builder`, `DateTimeFormatterBuilder`, `HttpRequest.newBuilder()`.
Da non confondere	un costruttore con dieci parametri senza nome o tanti `set` su un oggetto già creato (che resta a metà e modificabile per sempre); il Decorator, dove le catene sono di oggetti uno dentro l'altro (`new A(new B(…))`), non di chiamate sullo stesso oggetto.

16.1 La struttura



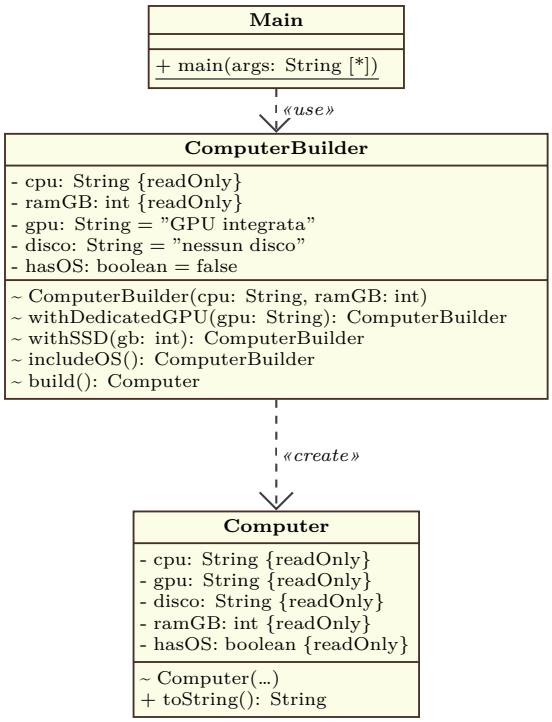
Builder: l'interfaccia dei passi di costruzione. **ConcreteBuilder**: li implementa, tiene il prodotto in costruzione e lo consegna alla fine (`getResult`, `build`). **Director**: conosce la sequenza dei passi e la ripete con qualunque builder. **Product**: l'oggetto complesso. `getResult()` sta solo nei builder concreti, perché ogni builder può produrre un tipo diverso.

16.2 L'esempio del corso: costruire un computer (senza Director)

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Computer gaming =
            new ComputerBuilder("AMD Ryzen 7", 32)
                .withDedicatedGPU("RX 7800")
                .withSSD(2000)
                .includeOS()
                .build();
        System.out.println(gaming);
        try {
            new ComputerBuilder("Intel Core i3", 2)
                .build();
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
class Computer {
    private final String cpu, gpu, disco;
    private final int ramGB;
    private final boolean hasOS;
    Computer(String cpu, int ramGB, String gpu,
             String disco, boolean hasOS) {
        this.cpu = cpu;     this.ramGB = ramGB;
        this.gpu = gpu;     this.disco = disco;
        this.hasOS = hasOS;
    }
    public String toString() {
        String os = hasOS ? ", con OS" : "";
        return cpu + ", " + ramGB + " GB, " + gpu
                + ", " + disco + os;
    }
}
class ComputerBuilder {
    private final String cpu;      // obbligatori
    private final int ramGB;
    private String gpu = "GPU integrata"; // opzioni
    private String disco = "nessun disco";
    private boolean hasOS = false;

    ComputerBuilder(String cpu, int ramGB) {
        this.cpu = cpu;
        this.ramGB = ramGB;
    }
    ComputerBuilder withDedicatedGPU(String gpu) {
        this.gpu = gpu;
        return this;
    }
    ComputerBuilder withSSD(int gb) {
        disco = "SSD " + gb + " GB";
        return this;
    }
    ComputerBuilder includeOS() {
        hasOS = true;
        return this;
    }
    Computer build() {
        if (ramGB < 4)
            throw new IllegalStateException(
                    "RAM insufficiente: min 4 GB");
        return new Computer(cpu, ramGB, gpu, disco,
                            hasOS);
    }
}
```



Terminale

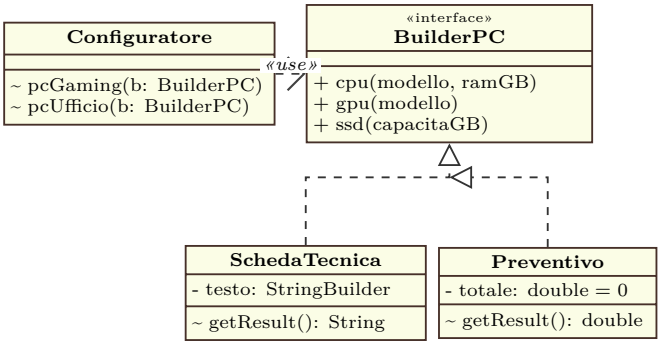
```
$ javac Main.java
$ java Main
AMD Ryzen 7, 32 GB, RX 7800, SSD 2000 GB, con OS
RAM insufficiente: min 4 GB
```

Ogni `with…` restituisce `this`: la catena lavora sempre sullo **stesso** builder. `build()` controlla i dati e solo alla fine crea il `Computer`, che non cambia più (campi `final`). Il secondo computer non viene creato: la regola sta nel builder, non nel prodotto.

16.3 Con il Director: stessa ricetta, prodotti diversi

Negozio.java

```java
public class Negozio {
    public static void main(String[] args) {
        Configuratore dir = new Configuratore();
        SchedaTecnica scheda = new SchedaTecnica();
        dir.pcGaming(scheda);      // stessa ricetta
        System.out.print(scheda.getResult());
        Preventivo prev = new Preventivo();
        dir.pcGaming(prev);        // altro prodotto
        System.out.println("Prezzo: "
                + prev.getResult());
    }
}
interface BuilderPC {                    // Builder
    void cpu(String modello, int ramGB);
    void gpu(String modello);
    void ssd(int capacitaGB);
}
class Configuratore {                    // Director
    void pcGaming(BuilderPC b) {
        b.cpu("AMD Ryzen 7", 32);
        b.gpu("AMD Radeon RX 7800");
        b.ssd(2000);
    }
    void pcUfficio(BuilderPC b) {
        b.cpu("Intel Core i3", 8);
        b.ssd(256);
    }
}
class SchedaTecnica implements BuilderPC {
    private final StringBuilder testo =
            new StringBuilder();
    public void cpu(String modello, int ramGB) {
        testo.append("CPU " + modello + ", RAM "
                + ramGB + " GB\n");
    }
    public void gpu(String modello) {
        testo.append("GPU " + modello + "\n");
    }
    public void ssd(int gb) {
        testo.append("SSD " + gb + " GB\n");
    }
    String getResult() { return testo.toString(); }
}
class Preventivo implements BuilderPC {
    private double totale = 0;
    public void cpu(String modello, int ramGB) {
        totale += 250 + 4 * ramGB;
    }
    public void gpu(String m) { totale += 550; }
    public void ssd(int gb) { totale += 0.08 * gb; }
    double getResult() { return totale; }
}
```



Terminale

```
$ javac Negozio.java
$ java Negozio
CPU AMD Ryzen 7, RAM 32 GB
GPU AMD Radeon RX 7800
SSD 2000 GB
Prezzo: 1088.0
```

Il **Director** (`Configuratore`) conosce le ricette ma non il prodotto; i due **ConcreteBuilder** eseguono gli stessi passi e producono una scheda (`String`) e un prezzo (`double`): per questo `getResult()` non sta nell'interfaccia. Prezzi di fantasia: 378 + 550 + 160 = 1088.

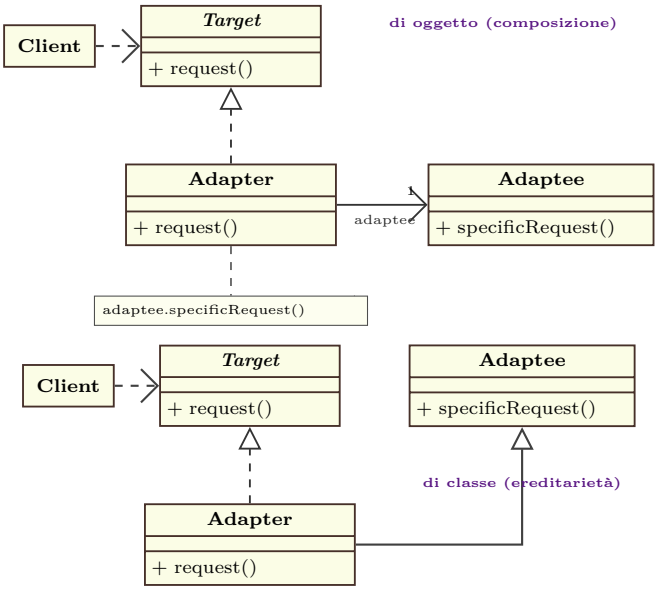
17 Adapter

Dispensa: capitolo 15. Pattern strutturale della GoF (sinonimo: Wrapper)

Carta d'identità

Intento	**Convertire l'interfaccia** di una classe esistente in quella che il client si aspetta, così che classi con interfacce incompatibili possano lavorare insieme. Come l'adattatore per le prese di corrente.
Nel testo	«riusare una classe (una libreria, un servizio, un microservizio) che esiste già», «la sua interfaccia non è quella che ci serve», «non possiamo modificarla» (non è nostra, o la usano altri), «unità di misura diverse» (libbre e chilogrammi), «formati diversi», «il nostro codice chiama `inEuro(…)`, il servizio offre `getRate(…)`».
Nel codice	una classe che `implements` l'interfaccia del client (Target) e ha un **campo del tipo della classe esistente** (Adaptee, di tipo **diverso** da Target), oppure la estende; i suoi metodi **traducono** parametri e risultati e chiamano un metodo con un altro nome o un'altra firma (`evaluate` chiama `calcolaIMC`).
Nel diagramma	`Client` → `«interface» Target`, realizzata da `Adapter`, che ha un'associazione verso `Adaptee` (adapter di oggetto) o ne è una sottoclasse (adapter di classe).
Varianti	adapter di **oggetto** (composizione: il più usato in Java, adatta anche le sottoclassi dell'Adaptee e lo nasconde) e adapter di **classe** (`extends Adaptee implements Target`: 17.3). L'uso tipico: un'interfaccia decisa dal nostro progetto davanti a un servizio esterno.
Da non confondere	il **Decorator** (19.1): avvolge un oggetto con la **stessa** interfaccia e aggiunge funzioni; l'Adapter avvolge un oggetto di un tipo **diverso** e ne cambia l'interfaccia. Il *Facade*: semplifica un intero sottosistema, non adatta una classe (24.2).

17.1 La struttura: adapter di oggetto e di classe

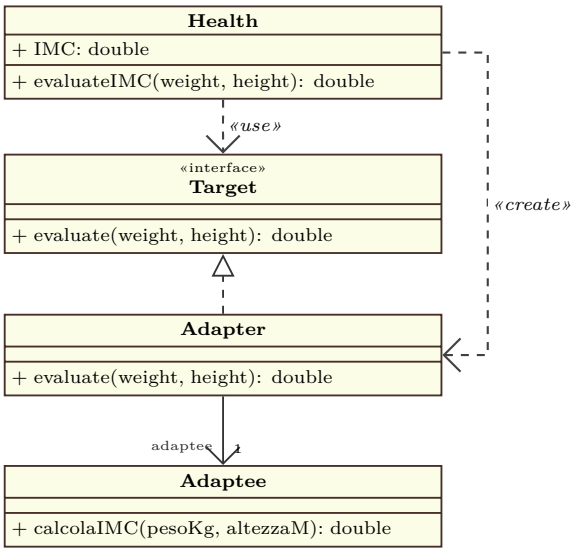


Client: vuole un servizio. **Target**: l'interfaccia che il client si aspetta. **Adaptee**: la classe esistente, con un'interfaccia diversa. **Adapter**: implementa Target e traduce le chiamate verso l'Adaptee, che **contiene** (sopra) o da cui **eredita** (sotto). Il client conosce solo Target: non sa che l'Adaptee esiste.

17.2 L'esempio del corso: l'indice di massa corporea

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Health health = new Health();
        health.evaluateIMC(165, 70); // lb, pollici
        System.out.println(health.IMC);
    }
}
class Health {                    // il Client
    public double IMC;
    public double evaluateIMC(double weight,
                              double height) {
        Target adapter = new Adapter();
        IMC = adapter.evaluate(weight, height);
        return IMC;
    }
}
interface Target {                // ciò che serve
    double evaluate(double weight, double height);
}
class Adapter implements Target {
    private final Adaptee adaptee = new Adaptee();
    public double evaluate(double weight,
                           double height) {
        // libbre -> chilogrammi, pollici -> metri
        return adaptee.calcolaIMC(weight * 0.453,
                                  height * 0.0254);
    }
}
class Adaptee {                   // ciò che esiste
    public double calcolaIMC(double pesoKg,
                             double altezzaM) {
        return pesoKg / (altezzaM * altezzaM);
    }
}
```



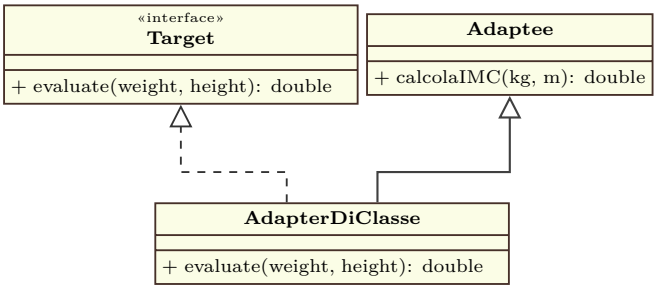
Terminale

```
$ javac Main.java
$ java Main
23.643873818359882
```

Tutta la «traduzione» sta nell'`Adapter`, in una riga: 165 libbre sono 74,7 kg, 70 pollici 1,78 m. `Health` conosce solo `Target`. Nella slide della soluzione l'adapter **divide** per 0,453 e 0,0254, e stampa un indice senza senso (0,000048): per passare dalle libbre ai chilogrammi si moltiplica. Nel codice del corso l'Adaptee usa a sua volta una classe `IMC` con lo stesso calcolo.

17.3 La variante: adapter di classe

```java
interface Target {
    double evaluate(double weight, double height);
}
class Adaptee {
    public double calcolaIMC(double kg, double m) {
        return kg / (m * m);
    }
}
// eredita dall'Adaptee invece di contenerlo
class AdapterDiClasse extends Adaptee
                      implements Target {
    public double evaluate(double weight,
                           double height) {
        return calcolaIMC(weight * 0.453,
                          height * 0.0254);
    }
}
```



L'adapter **è** un Adaptee e chiama il metodo ereditato. Più compatto, ma legato a **una** classe Adaptee, che espone tutta (chi ha un `AdapterDiClasse` può chiamare anche `calcolaIMC`); e in Java si estende una classe sola. Per questo si preferisce l'adapter di oggetto.

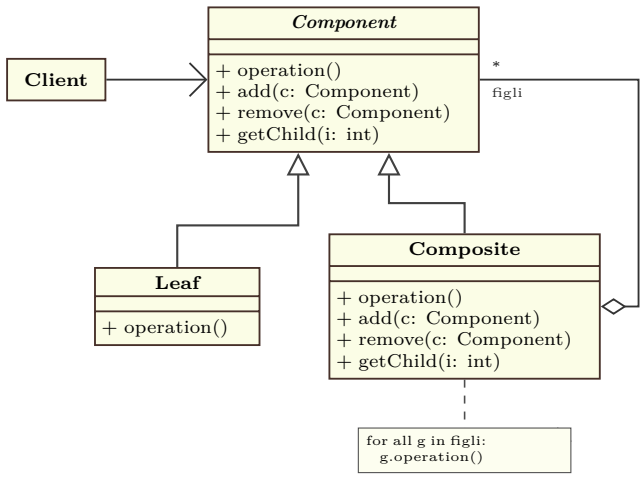
18 Composite

Dispensa: capitolo 15. Pattern strutturale della GoF

Carta d'identità

Intento	Comporre gli oggetti in **strutture ad albero** (gerarchie tutto-parte) e permettere al client di trattare **allo stesso modo** oggetti singoli e oggetti composti.
Nel testo	«gerarchia», «albero», «un elemento può contenere altri elementi, che a loro volta…», «cartelle e file», «menu e sottomenu», «gruppi e sottogruppi», «pannelli che contengono pulsanti e altri pannelli», «un'operazione sul tutto si applica a tutte le parti», «trattare in modo uniforme».
Nel codice	un'interfaccia (o classe astratta) Component; una classe **foglia** che la implementa; una classe **composta** che la implementa **e** ha una **collezione dello stesso tipo** (`List<AbstractFile>`), con `add` e `remove`; il metodo del composto chiama lo **stesso metodo** su tutti i figli (ricorsione), e il client lo chiama una volta sola, sulla radice.
Nel diagramma	`Leaf` e `Composite` realizzano (o estendono) `Component`; da `Composite` parte un'**aggregazione con molteplicità** `*` che torna verso `Component`: un composto contiene componenti, quindi anche altri composti.
Varianti	**trasparenza**: `add`, `remove`, `getChild` dichiarati nel Component (il client tratta tutti allo stesso modo, ma deve gestire `add` su una foglia, per esempio con un'eccezione) oppure **sicurezza**: solo nel Composite (il compilatore impedisce di aggiungere figli a una foglia). Nelle GUI di Java: un `Container` è un `Component` che contiene `Component`.
Da non confondere	il **Decorator** (19.1): anche lui implementa il Component e ne contiene uno, ma **uno solo**, e aggiunge comportamento; il Composite ne contiene **molti** e combina i risultati. Una semplice associazione riflessiva (`List<Persona> figli`): senza un'interfaccia comune fra foglie e composti non è un Composite.

18.1 La struttura

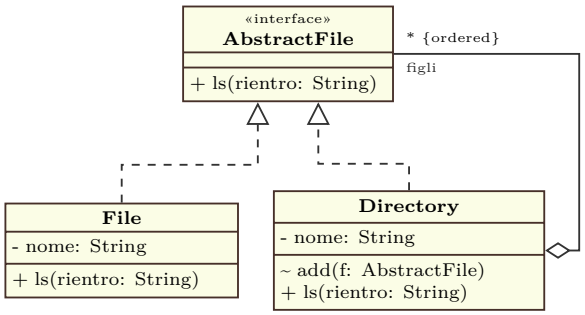


Component: l'interfaccia comune, con le operazioni e (nella forma trasparente) la gestione dei figli. **Leaf**: un oggetto senza figli. **Composite**: memorizza i figli e implementa le operazioni chiamandole su di loro. **Client**: lavora solo con il Component, senza sapere se ha in mano una foglia o un composto. La relazione decisiva è l'aggregazione **dal Composite al Component**: è lei che rende l'albero ricorsivo.

18.2 L'esempio del corso: il file system

EsempioComposite.java

```java
public class EsempioComposite {
    public static void main(String[] args) {
        Directory one = new Directory("dir111"),
                  two = new Directory("dir222"),
                  thr = new Directory("dir333");
        one.add(new File("a"));
        one.add(two);
        one.add(new File("b"));
        two.add(new File("c"));
        two.add(new File("d"));
        two.add(thr);
        thr.add(new File("e"));
        one.ls("");           // una sola chiamata
    }
}
interface AbstractFile {            // Component
    void ls(String rientro);
}
class File implements AbstractFile { // Leaf
    private final String nome;
    File(String nome) { this.nome = nome; }
    public void ls(String rientro) {
        System.out.println(rientro + nome);
    }
}
// Composite
class Directory implements AbstractFile {
    private final String nome;
    private final List<AbstractFile> figli =
            new ArrayList<>();
    Directory(String nome) { this.nome = nome; }
    void add(AbstractFile f) { figli.add(f); }
    public void ls(String rientro) {
        System.out.println(rientro + nome);
        for (AbstractFile f : figli)
            f.ls(rientro + "   ");   // ricorsione
    }
}
```



Terminale

```
$ javac EsempioComposite.java
$ java EsempioComposite
dir111
   a
   dir222
      c
      d
      dir333
         e
   b
```

`Directory` non sa se un figlio è un file o un'altra directory: chiama `ls` e ci pensa il **polimorfismo**. Il `main` chiama `ls` una volta, sulla radice, e la ricorsione visita l'albero. Nel codice del corso il rientro è una `StringBuffer` statica che ogni `ls` allunga e poi accorcia, e `add` riceve un `Object`; qui il rientro è un parametro e `add` accetta solo `AbstractFile`.

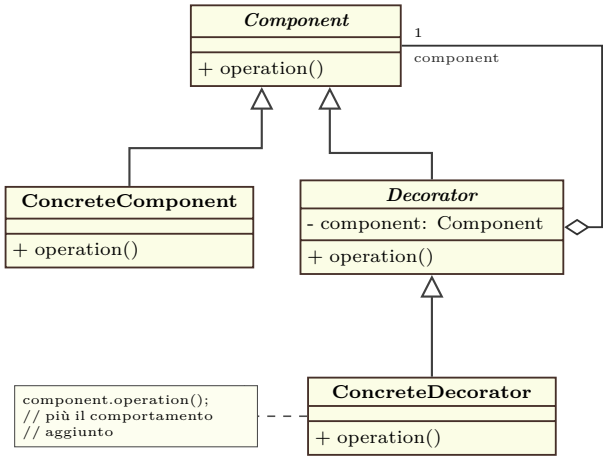
19 Decorator

Dispensa: capitolo 15. Pattern strutturale della GoF

Carta d'identità

Intento	Aggiungere **dinamicamente** responsabilità a un oggetto, **avvolgendolo** in un altro oggetto con la **stessa interfaccia**: un'alternativa flessibile alle sottoclassi.
Nel testo	«aggiungere funzionalità (ingredienti, optional, opzioni, filtri) a un oggetto», «a tempo di esecuzione», «in qualunque combinazione, anche ripetuta» (la doppia mozzarella), «senza una sottoclasse per ogni combinazione» (con 10 ingredienti sarebbero 2^{10} classi), «un flusso di dati con buffer, cifratura, compressione».
Nel codice	una classe (spesso astratta) che `implements` il **Component** e ha **un** campo dello **stesso tipo**, ricevuto nel costruttore (`protected Pizza pizza`); i decoratori concreti chiamano il metodo dell'oggetto avvolto e **aggiungono** qualcosa (`pizza.getCost() + 1.50`); il client scrive catene come `new Mozzarella(new Fungo(new OlioEPomodoro()))`.
Nel diagramma	`Decorator` realizza (o estende) `Component` e ha un'aggregazione verso `Component` con molteplicità `1`; i `ConcreteDecorator` estendono il Decorator; il `ConcreteComponent` realizza il Component.
Varianti	il Decorator come classe astratta che delega tutto per default (i decoratori concreti ridefiniscono solo ciò che cambiano); nella libreria di Java i flussi di I/O: `new BufferedReader(new InputStreamReader(System.in))` (byte → caratteri → righe).
Da non confondere	il **Composite**: contiene **molti** componenti, il Decorator **uno**; l'**Adapter**: l'oggetto avvolto ha un'interfaccia **diversa** e l'adapter la traduce, il Decorator ha la **stessa** interfaccia e aggiunge; l'ereditarietà: aggiunge funzioni in modo statico, nel codice, non a tempo di esecuzione.

19.1 La struttura

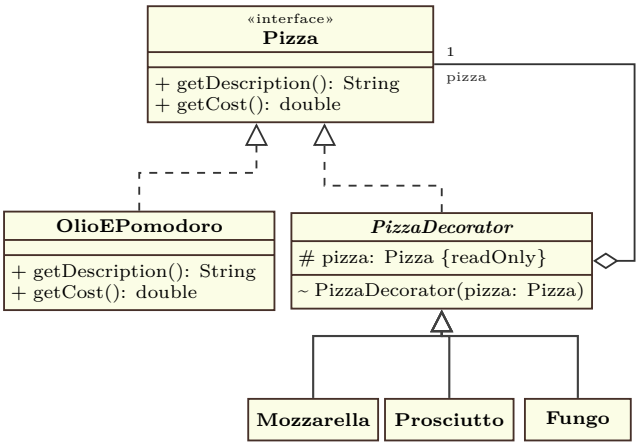


Component: l'interfaccia comune.
ConcreteComponent: l'oggetto di base. **Decorator**: **è** un Component (eredita l'interfaccia) e **ha** un Component (il campo a cui delega): è questa doppia relazione che permette di avvolgere un decoratore dentro un altro.
ConcreteDecorator: chiama l'oggetto avvolto e aggiunge il suo pezzo. La chiamata scende lungo la catena fino all'oggetto di base, e i risultati risalgono.

19.2 L'esempio del corso: la pizza

Pizzeria.java

```java
public class Pizzeria {
    public static void main(String[] args) {
        Pizza p = new OlioEPomodoro();
        stampa(p);
        p = new Mozzarella(p); // ogni new avvolge
        p = new Fungo(p);      // la pizza di prima
        p = new Mozzarella(p); // doppia mozzarella
        stampa(p);
    }
    static void stampa(Pizza p) {
        System.out.println(p.getDescription());
        System.out.println("  " + p.getCost());
    }
}
interface Pizza {                    // Component
    String getDescription();
    double getCost();
}
class OlioEPomodoro implements Pizza {   // di base
    public String getDescription() {
        return "Olio e Pomodoro";
    }
    public double getCost() { return 3.00; }
}
abstract class PizzaDecorator implements Pizza {
    protected final Pizza pizza;         // avvolta
    PizzaDecorator(Pizza pizza) {
        this.pizza = pizza;
    }
}
class Mozzarella extends PizzaDecorator {
    Mozzarella(Pizza p) { super(p); }
    public String getDescription() {
        return pizza.getDescription()
                + ", Mozzarella";
    }
    public double getCost() {
        return pizza.getCost() + 1.50;
    }
}
class Prosciutto extends PizzaDecorator {
    Prosciutto(Pizza p) { super(p); }
    public String getDescription() {
        return pizza.getDescription()
                + ", Prosciutto";
    }
    public double getCost() {
        return pizza.getCost() + 1.00;
    }
}
class Fungo extends PizzaDecorator {
    Fungo(Pizza p) { super(p); }
    public String getDescription() {
        return pizza.getDescription()
                + ", Funghi";
    }
    public double getCost() {
        return pizza.getCost() + 2.00;
    }
}
```



Terminale

```
$ javac Pizzeria.java
$ java Pizzeria
Olio e Pomodoro
  3.0
Olio e Pomodoro, Mozzarella, Funghi, Mozzarella
  8.0
```

`getCost()` sull'anello più esterno scende fino alla pizza base, che risponde 3,00; poi i risultati risalgono e ogni decoratore aggiunge il suo pezzo: $3 + 1{,}5 + 2 + 1{,}5 = 8$. Le combinazioni (anche ripetute) si scelgono a tempo di esecuzione con una classe per ingrediente. Il prezzo da pagare: ogni chiamata attraversa tutta la catena.

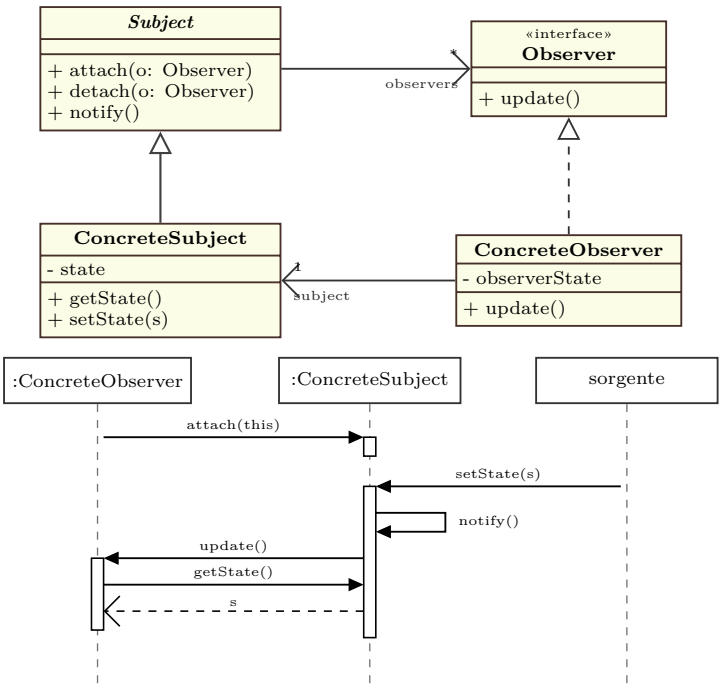
20 Observer

Dispensa: capitolo 15. Pattern comportamentale della GoF

Carta d'identità

Intento	Definire una dipendenza **1:N** fra oggetti: quando il **soggetto** cambia stato, tutti gli **osservatori** vengono avvisati e aggiornati **automaticamente**.
Nel testo	«quando X cambia, Y deve aggiornarsi automaticamente», «notificare», «avvisare tutti gli interessati», «iscriversi» e «cancellarsi», «abbonati», «più viste degli stessi dati», «osservatori di tipi diversi, aggiunti mentre il programma gira», «un sensore o un servizio fornisce un dato aggiornato», «eventi dell'interfaccia grafica»; nelle slide: aggiornare il valore delle azioni quando cambia la quotazione della società.
Nel codice	il soggetto ha una **lista di un'interfaccia** (`List<Osservatore>`), un metodo per **registrare** e uno per **togliere** gli osservatori (`attach`/`detach`, `add…Listener`), e un metodo di **notifica** che percorre la lista chiamando `update()` su tutti; il metodo che cambia lo stato (`setStato`) chiama la notifica. L'osservatore implementa l'interfaccia, ha un riferimento al soggetto e nel suo `update()` ne legge il nuovo stato.
Nel diagramma	`Subject` → `«interface» Observer` con molteplicità `*` (ruolo `observers`); `ConcreteObserver` realizza Observer e ha un'associazione `1` verso `ConcreteSubject`; spesso la nota `for all o in observers: o.update()`.
Varianti	soggetto astratto (classe con la lista) o interfaccia; i *listener* di Swing con una *lambda* (`pulsante.addActionListener(e -> …)`); il modello *pull* (l'osservatore chiede lo stato, come qui) o *push* (il soggetto passa il dato a `update(valore)`) oltre le slide; il cuore dell'MVC (la View osserva il Model). `java.util.Observable` esiste ma è deprecata da Java 9. In Java il metodo non può chiamarsi `notify()`: esiste già, `final`, in `Object`.
Da non confondere	il **Composite**: anche il composto ha una lista e chiama un metodo su tutti, ma i figli hanno la **stessa interfaccia** del composto e formano un albero; lo **Strategy**: il contesto ha **un** oggetto di un'interfaccia, non una lista, e gli delega un calcolo.

20.1 La struttura e il diagramma di sequenza



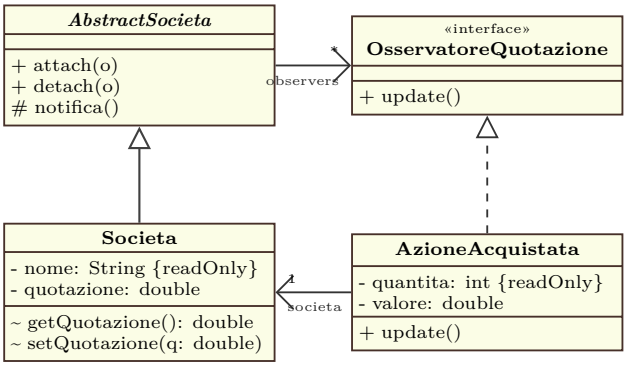
Subject: tiene la lista degli osservatori e li avvisa (`notify`). **Observer**: l'interfaccia con `update()`. **ConcreteSubject**: ha lo stato; quando cambia, chiama `notify`. **ConcreteObserver**: conosce il soggetto e, ricevuto `update()`, gli chiede il nuovo stato.

Sotto, la sequenza: l'osservatore si registra; più tardi una sorgente esterna cambia lo stato; il soggetto notifica tutti, e ogni osservatore rilegge lo stato. Il soggetto conosce gli osservatori **solo** attraverso l'interfaccia: se ne possono aggiungere di nuovi senza toccarlo.

20.2 Il problema delle slide: il valore delle azioni

Portafoglio.java

```java
public class Portafoglio {
    public static void main(String[] args) {
        Societa acme = new Societa("ACME", 10.0);
        AzioneAcquistata mie =
                new AzioneAcquistata(acme, 100);
        AzioneAcquistata tue =
                new AzioneAcquistata(acme, 40);
        System.out.println(mie + "\n" + tue);
        acme.setQuotazione(12.5); // la borsa cambia
        System.out.println(mie + "\n" + tue);
    }
}
interface OsservatoreQuotazione {       // Observer
    void update();
}
abstract class AbstractSocieta {        // Subject
    private final List<OsservatoreQuotazione>
            observers = new ArrayList<>();
    public void attach(OsservatoreQuotazione o) {
        observers.add(o);
    }
    public void detach(OsservatoreQuotazione o) {
        observers.remove(o);
    }
    protected void notifica() {         // il notify
        for (OsservatoreQuotazione o : observers)
            o.update();
    }
}
class Societa extends AbstractSocieta {
    private final String nome;
    private double quotazione;
    Societa(String nome, double quotazione) {
        this.nome = nome;
        this.quotazione = quotazione;
    }
    String getNome() { return nome; }
    double getQuotazione() { return quotazione; }
    void setQuotazione(double q) {
        quotazione = q;
        notifica();     // avvisa gli osservatori
    }
}
class AzioneAcquistata
        implements OsservatoreQuotazione {
    private final Societa societa;
    private final int quantita;
    private double valore;
    AzioneAcquistata(Societa s, int quantita) {
        societa = s;
        this.quantita = quantita;
        s.attach(this);           // si iscrive
        update();                 // valore iniziale
    }
    public void update() {
        valore = societa.getQuotazione() * quantita;
    }
    public String toString() {
        return quantita + " azioni "
                + societa.getNome() + " = " + valore
                + " euro";
    }
}
```



Terminale

```
$ javac Portafoglio.java
$ java Portafoglio
100 azioni ACME = 1000.0 euro
40 azioni ACME = 400.0 euro
100 azioni ACME = 1250.0 euro
40 azioni ACME = 500.0 euro
```

Il `main` non chiede mai alle azioni di ricalcolarsi: cambia la quotazione, e le due azioni lo vengono a sapere da sole. `AzioneAcquistata` si iscrive nel costruttore (`attach`); `setQuotazione` è un setter con una riga in più, la notifica. Le slide danno il diagramma; il codice è della dispensa. L'esempio di codice delle slide ha la stessa forma, con `Soggetto`, `Osservatore` e `aggiorna()`.

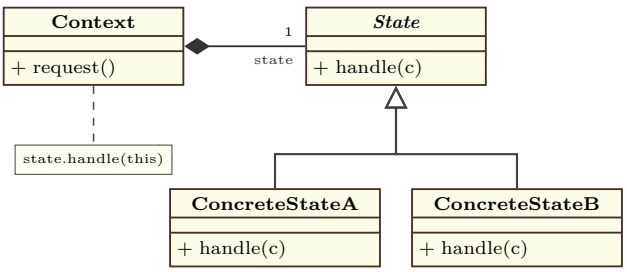
21 State

Dispensa: capitoli 9 e 15. Pattern comportamentale della GoF

Carta d'identità

Intento	Permettere a un oggetto di **cambiare comportamento quando cambia il suo stato interno**: sembra che cambi classe. È il modo a oggetti di realizzare un **diagramma di stato**.
Nel testo	«si comporta in modo diverso a seconda dello stato (della fase)», «lo stesso comando ha effetti diversi», un **diagramma di stato** da implementare, «stati e transizioni»; gli esempi tipici: il **distributore automatico**, un ordine (in preparazione, spedito, consegnato), una connessione (aperta, chiusa), un'interfaccia grafica con più schermate.
Nel codice	il **contesto** ha un campo di un tipo astratto (`State stato`) e **delega** ogni evento allo stato corrente passandogli se stesso: `stato.inserisciMoneta(this)`; una **sottoclasse per stato**, con un metodo per evento; le **transizioni le fanno gli stati stessi**, chiamando `contesto.setState(…)`; spesso gli stati sono oggetti condivisi, creati una volta in campi statici.
Nel diagramma	`Context` con una composizione (o aggregazione) `1` verso la classe astratta `State`, e le sottoclassi `ConcreteStateA`, `ConcreteStateB`…; la nota `state.handle()` sul contesto.
Varianti	`State` classe astratta con i metodi vuoti (un evento non previsto viene ignorato) o interfaccia; stati condivisi e senza campi (i dati stanno nel contesto) oppure uno stato per contesto, se lo stato deve ricordare qualcosa; senza pattern, gli `switch` annidati (11.9).
Da non confondere	lo **Strategy** (22.1): **stessa struttura**, ma l'oggetto lo sceglie il **client** e le strategie non si conoscono fra loro; nello State sono gli **stati** a decidere il prossimo stato, e ognuno sa in quali altri si può passare.

21.1 La struttura e la corrispondenza con il diagramma di stato



Context: l'oggetto che il client usa; tiene lo stato corrente e i dati. **State**: un metodo per ogni evento, vuoti. **ConcreteState**: uno per stato del diagramma; ridefinisce solo gli eventi che lì hanno un effetto.

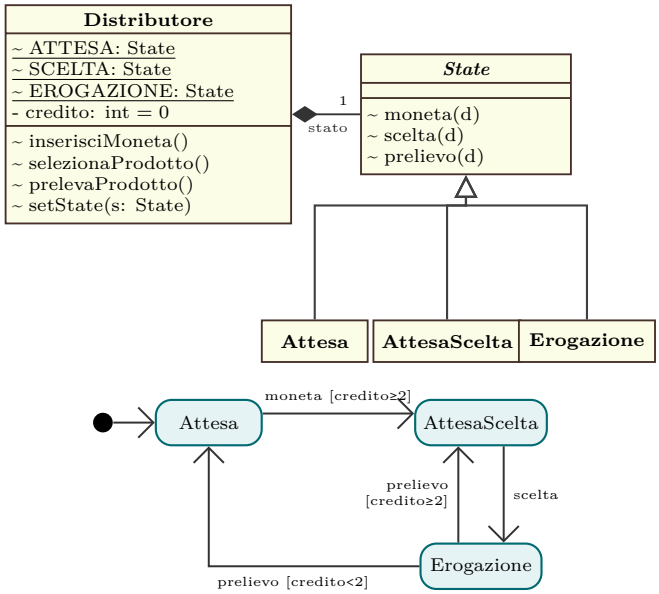
Diagramma di stato	Codice
stato	sottoclasse di `State`
evento	metodo di `State`
guardia	`if` nel metodo
azione	corpo del metodo
transizione	`setState(…)`

21.2 L'esempio del corso: il distributore automatico (ridotto)

Cliente.java

```java
public class Cliente {
    public static void main(String[] args) {
        Distributore d = new Distributore();
        d.selezionaProdotto();
        d.inserisciMoneta();
        d.inserisciMoneta();
        d.selezionaProdotto();
        d.inserisciMoneta();
        d.prelevaProdotto();
    }
}
class Distributore {                        // Context
    static final State ATTESA = new Attesa(),
            SCELTA = new AttesaScelta(),
            EROGAZIONE = new Erogazione();
    private State stato = ATTESA;
    private int credito = 0;

    void setState(State s) { stato = s; }
    int getCredito() { return credito; }
    void setCredito(int c) { credito = c; }
    // ogni evento è delegato allo stato corrente
    void inserisciMoneta() { stato.moneta(this); }
    void selezionaProdotto() { stato.scelta(this); }
    void prelevaProdotto() { stato.prelievo(this); }
}
abstract class State {              // eventi: vuoti
    void moneta(Distributore d) { }
    void scelta(Distributore d) { }
    void prelievo(Distributore d) { }
}
class Attesa extends State {
    void moneta(Distributore d) {
        d.setCredito(d.getCredito() + 1);
        System.out.println("Credito: "
                + d.getCredito());
        if (d.getCredito() >= 2)        // guardia
            d.setState(Distributore.SCELTA);
    }
    void scelta(Distributore d) {
        System.out.println("Credito insufficiente");
    }
}
class AttesaScelta extends State {
    void moneta(Distributore d) {
        d.setCredito(d.getCredito() + 1);
        System.out.println("Credito: "
                + d.getCredito());
    }
    void scelta(Distributore d) {
        System.out.println("Erogazione...");
        d.setState(Distributore.EROGAZIONE);
    }
}
class Erogazione extends State {
    void moneta(Distributore d) {
        System.out.println("Attendi l'erogazione");
    }
    void prelievo(Distributore d) {
        d.setCredito(d.getCredito() - 2);
        System.out.println("Ecco il prodotto, "
                + "credito: " + d.getCredito());
        if (d.getCredito() >= 2)
            d.setState(Distributore.SCELTA);
        else
            d.setState(Distributore.ATTESA);
    }
}
```



Terminale

```
$ javac Cliente.java
$ java Cliente
Credito insufficiente
Credito: 1
Credito: 2
Erogazione...
Attendi l'erogazione
Ecco il prodotto, credito: 0
```

La stessa chiamata (`selezionaProdotto()`) fa cose diverse a seconda dello stato: in Attesa rifiuta, in AttesaScelta avvia l'erogazione. Una **transizione** non crea oggetti: `setState` sposta il riferimento fra tre oggetti stato creati una volta sola, senza campi (il credito sta nel distributore). Il codice del corso ha più eventi (restituzione del credito, time out, spegnimento).

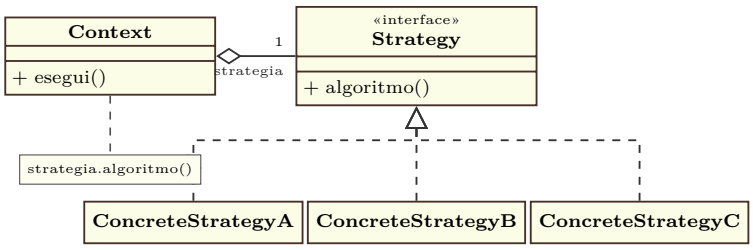
22 Strategy

Dispensa: capitolo 15. Pattern comportamentale della GoF

Carta d'identità

Intento	Definire una **famiglia di algoritmi**, incapsularli uno per classe e renderli **intercambiabili**: l'algoritmo può variare indipendentemente da chi lo usa.
Nel testo	«modi diversi di calcolare (spedire, pagare, ordinare, scontare, validare)», «politiche», «il cliente sceglie il metodo di spedizione», «cambiare l'algoritmo a tempo di esecuzione», «aggiungere un nuovo modo di calcolare senza modificare la classe che lo usa».
Nel codice	un'interfaccia con **un** metodo (o pochi), che è l'algoritmo (`double costo(double pesoKg)`); più classi che la implementano, ognuna con la sua formula; il **contesto** ha un campo di quel tipo, **impostato dal client** (costruttore o `setSpedizione(…)`), e gli delega il calcolo. Spesso al posto delle classi c'è una *lambda* o un `Comparator`.
Nel diagramma	`Context` con un'aggregazione verso `«interface» Strategy`, realizzata da `ConcreteStrategyA`, `B`, `C`: la stessa forma dello State.
Varianti	la strategia nel costruttore (fissa) o in un setter (cambia quando il client vuole); con una lambda (`ordine.setSpedizione(peso -> 0.0)`); nella libreria di Java: `lista.sort(comparatore)`, dove il `Comparator` è la strategia di confronto.
Da non confondere	lo **State** (21.1): stessa struttura, ma lì l'oggetto cambia **da solo**, perché gli stati chiamano `setState`; la **Dependency Injection** (23.1): dice **come** arriva l'oggetto (da fuori), non che cosa rappresenta; una strategia di solito arriva proprio per iniezione.

22.1 La struttura



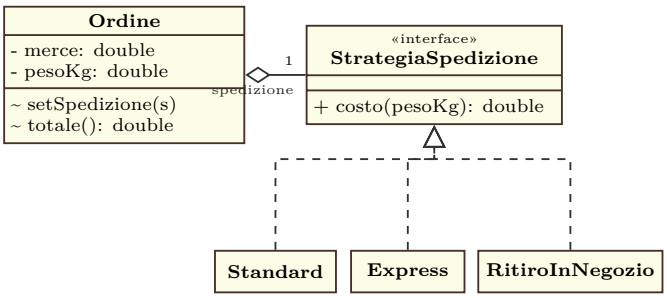
Strategy: l'interfaccia comune degli algoritmi. **ConcreteStrategy**: un algoritmo ciascuna; non si conoscono fra loro. **Context**: usa una strategia senza sapere quale; la sceglie il **client**.

	State	**Strategy**
chi cambia l'oggetto	gli stati	il client
si conoscono?	sì	no
rappresenta	una fase	un modo di fare

22.2 L'esempio della dispensa: il costo della spedizione

Negozio.java

```java
public class Negozio {
    public static void main(String[] args) {
        Ordine ordine = new Ordine(40.0, 6.0);
        System.out.println("Standard:   "
                + ordine.totale());
        ordine.setSpedizione(new Express());
        System.out.println("Express:    "
                + ordine.totale());
        ordine.setSpedizione(new RitiroInNegozio());
        System.out.println("In negozio: "
                + ordine.totale());
    }
}
interface StrategiaSpedizione {          // Strategy
    double costo(double pesoKg);
}
class Standard implements StrategiaSpedizione {
    public double costo(double pesoKg) {
        return 5.0 + 0.5 * pesoKg;
    }
}
class Express implements StrategiaSpedizione {
    public double costo(double pesoKg) {
        return 12.0 + 1.0 * pesoKg;
    }
}
class RitiroInNegozio
        implements StrategiaSpedizione {
    public double costo(double pesoKg) {
        return 0.0;
    }
}
class Ordine {                           // Context
    private final double merce, pesoKg;
    private StrategiaSpedizione spedizione =
            new Standard();
    Ordine(double merce, double pesoKg) {
        this.merce = merce;
        this.pesoKg = pesoKg;
    }
    void setSpedizione(StrategiaSpedizione s) {
        spedizione = s;      // la sceglie il client
    }
    double totale() {
        return merce + spedizione.costo(pesoKg);
    }
}
```



Terminale

```
$ javac Negozio.java
$ java Negozio
Standard:   48.0
Express:    58.0
In negozio: 40.0
```

Lo stesso ordine (40 euro di merce, 6 kg) cambia algoritmo tre volte, e `Ordine` non cambia mai: per una nuova spedizione basta una classe nuova. Le slide nominano lo Strategy nella tabella dei pattern senza un esempio; questo è quello della dispensa.

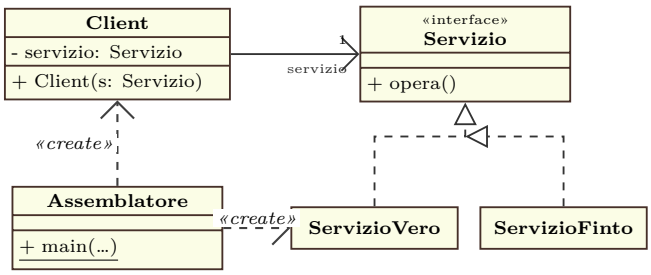
23 Dependency Injection

Dispensa: capitolo 15. Un pattern moderno, fuori dal catalogo della GoF

Carta d'identità

Intento	Le risorse da cui un oggetto dipende gli vengono **passate dall'esterno** (iniettate), di solito nel costruttore, invece di essere create dall'oggetto stesso. Si chiama anche **inversione del controllo**: prima si crea la dipendenza, poi chi la usa.
Nel testo	«testare la classe senza il servizio reale» (che non è disponibile, è lento o non è ancora pronto), «un oggetto **fittizio** (finto, *mock*)», «cambiare database o servizio senza modificare la classe», «configurare dall'esterno quale implementazione usare», «Spring», «`@Autowired`», «`@Inject`».
Nel codice	un costruttore che riceve un'**interfaccia** (`AllarmeCaldo(ServizioMeteo meteo)`) e la salva in un campo `final`; dentro la classe **nessun `new`** della classe concreta; chi crea l'oggetto (il `main`, un test, un framework) crea **prima** la dipendenza e poi la passa.
Nel diagramma	la classe con un'associazione verso un'`«interface»`, realizzata da più classi (quella vera e quella finta); le dipendenze «create» stanno sul `main` o sul test, non sulla classe.
Varianti	iniezione nel **costruttore** (la più comune) o con un **setter** oltre le slide; a mano, nel `main`, o con un framework (Spring, Jakarta EE) che crea gli oggetti e li collega. Nella dispensa: la `Cassa` con il `GatewayPagamenti` (capitolo 4), il `Controller` con il `ListinoDAO` (capitolo 8).
Da non confondere	il **Singleton** (15.1): lì chi usa l'oggetto **va a prenderselo** (`getInstance()`), qui gli viene **dato**; lo **Strategy**: è spesso iniettato, ma dice che cosa è l'oggetto (un algoritmo intercambiabile), non come arriva.

23.1 La struttura

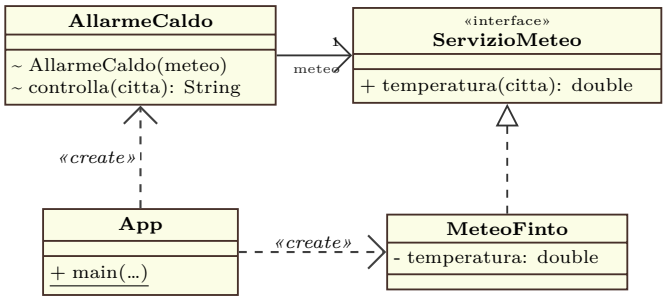


Client: dichiara nel costruttore di che cosa ha bisogno, con il tipo di un'interfaccia. **Servizio**: l'interfaccia. **ServizioVero**, **ServizioFinto**: le implementazioni, quella di produzione e quella per i test. **Assemblatore**: il `main`, il test o il framework, l'unico che conosce le classi concrete. Resta una dipendenza dal **tipo** della risorsa, e chi crea l'oggetto deve conoscere tutte le sue dipendenze.

23.2 L'esempio della dispensa: un allarme provato con un servizio finto

App.java

```java
public class App {
    public static void main(String[] args) {
        // prima la dipendenza, poi chi la usa
        ServizioMeteo finto = new MeteoFinto(38.0);
        AllarmeCaldo a = new AllarmeCaldo(finto);
        System.out.println(a.controlla("Napoli"));
        a = new AllarmeCaldo(new MeteoFinto(22.5));
        System.out.println(a.controlla("Napoli"));
    }
}
interface ServizioMeteo {
    double temperatura(String citta);   // gradi C
}
class AllarmeCaldo {
    private final ServizioMeteo meteo; // risorsa
    AllarmeCaldo(ServizioMeteo meteo) {
        this.meteo = meteo;             // iniettata
    }
    String controlla(String citta) {
        double t = meteo.temperatura(citta);
        if (t >= 35)
            return "ALLARME: a " + citta
                   + " ci sono " + t + " gradi";
        return "Nessun allarme a " + citta;
    }
}
// oggetto fittizio per i test: risponde sempre
// con la temperatura che gli si dà
class MeteoFinto implements ServizioMeteo {
    private final double temperatura;
    MeteoFinto(double t) { temperatura = t; }
    public double temperatura(String citta) {
        return temperatura;
    }
}
```



Terminale

```
$ javac App.java
$ java App
ALLARME: a Napoli ci sono 38.0 gradi
Nessun allarme a Napoli
```

`AllarmeCaldo` è la stessa classe in produzione e nei test: in produzione riceve `MeteoHttp`, che interroga un servizio REST (nella dispensa), nei test `MeteoFinto`, che risponde subito e con la temperatura scelta dal test. Il controesempio: `private final ServizioMeteo meteo = new MeteoHttp(…);` dentro la classe lega l'allarme al servizio vero, e nessun test lo può sostituire.

24 Pattern a confronto

Dispensa: capitolo 15. Quelli che si somigliano, e gli altri della GoF nominati nelle slide

24.1 I pattern che si somigliano

State o Strategy?	Stessa struttura (un contesto con un campo di tipo astratto e più implementazioni). Nello **State** sono gli **stati** a cambiare l'oggetto puntato (`setState`), si conoscono fra loro e rappresentano **fasi** della vita del contesto; nello **Strategy** lo sceglie il **client**, le strategie non si conoscono e sono **modi diversi** di fare la stessa cosa.
Decorator o Composite?	Entrambi implementano il Component **e** ne contengono. Il **Decorator** ne contiene **uno** e **aggiunge** comportamento (una catena); il **Composite** ne contiene **molti** e **combina** i risultati dei figli (un albero).
Decorator o Adapter?	Entrambi avvolgono un oggetto e gli delegano il lavoro. Il **Decorator** ha la **stessa** interfaccia dell'oggetto avvolto e aggiunge funzioni; l'**Adapter** avvolge un oggetto con un'interfaccia **diversa** e la traduce, senza aggiungere nulla.
Decorator o Proxy? oltre le slide	Stessa forma. Il Decorator **aggiunge** responsabilità e si concatena; il Proxy **controlla l'accesso** all'oggetto vero (permessi, cache, creazione pigra) e di solito è uno solo.
Adapter o Facade? oltre le slide	L'Adapter adatta **una** classe a un'interfaccia che esiste già; la Facade inventa un'interfaccia **semplice** davanti a un intero sottosistema di classi.
Observer o Composite?	Entrambi hanno una lista e chiamano un metodo su tutti. Gli **osservatori** hanno un'interfaccia diversa da quella del soggetto e vengono avvisati di un cambiamento; i **figli** di un composto hanno la sua stessa interfaccia e formano un albero.
Builder o costruttore?	Il costruttore riceve tutto in una volta (e con molti parametri facoltativi diventa illeggibile); i `set` lasciano l'oggetto a metà. Il **Builder** raccoglie i dati a passi e crea il prodotto, già controllato e immutabile, solo con `build()`.
Singleton, classe statica o DI?	La classe con soli metodi `static` non ha un oggetto; il **Singleton** ha un oggetto unico che chi lo usa **va a prendere** (`getInstance()`); con la **Dependency Injection** l'oggetto (unico o no) viene **passato** a chi lo usa, e nei test si sostituisce.
Strategy o DI?	Non si escludono: lo Strategy dice **che cosa** è l'oggetto (un algoritmo intercambiabile), la DI dice **come arriva** (da fuori, nel costruttore). Una strategia iniettata nel costruttore è entrambe le cose.

24.2 Gli altri pattern della GoF, nominati nelle slide

Le slide li elencano con la loro definizione (seconda colonna); come riconoscerli nel codice (terza colonna) è un'aggiunta oltre le slide, per non restare senza parole se un esercizio ne mostra uno.

Pattern	Che cosa fa	Come si riconosce nel codice
Abstract Factory	Un'interfaccia per creare **famiglie** di oggetti correlati senza specificarne le classi concrete	un'interfaccia con più metodi `creaX()`, `creaY()`; una fabbrica concreta per famiglia (`TemaChiaro`, `TemaScuro`); il client riceve la fabbrica e non fa `new`
Factory Method	Un'interfaccia per creare un oggetto, ma le **sottoclassi** scelgono la classe da istanziare	un metodo astratto `creaProdotto()` nella superclasse, ridefinito nelle sottoclassi; la superclasse usa il prodotto senza sapere quale sia
Prototype	Crea i nuovi oggetti **copiando** un'istanza prototipo	`clone()` o `copia()` chiamato su un oggetto esistente al posto di `new`; spesso un registro di prototipi
Bridge	**Disaccoppia** un'astrazione dalla sua implementazione, così che possano variare indipendentemente	due gerarchie parallele; le classi della prima hanno un campo del tipo della seconda (una `Forma` con un `Disegnatore`)
Facade	Un'interfaccia **unificata** a un sottosistema, più facile da usare	una classe con pochi metodi semplici, ognuno dei quali chiama molte classi del sottosistema
Proxy	Un **surrogato** di un oggetto, per controllarne l'accesso	stessa interfaccia dell'oggetto vero, un campo verso di lui, controlli prima di delegare (permessi, cache, caricamento pigro)
Flyweight	La **condivisione** per gestire in modo efficiente molti oggetti piccoli	una fabbrica con una mappa che restituisce oggetti immutabili già creati invece di crearne di nuovi
Template Method	Lo **scheletro** di un algoritmo, con alcuni passi definiti dalle sottoclassi	un metodo (spesso `final`) nella superclasse astratta che chiama, in ordine, metodi astratti o `protected` ridefiniti nelle sottoclassi
Visitor	Un'operazione sugli elementi di una struttura; se ne aggiungono senza modificare le classi degli elementi	`accept(Visitor v)` negli elementi, che chiama `v.visit(this)`; un'interfaccia `Visitor` con un `visit` per ogni tipo di elemento
Iterator	Accesso in sequenza agli elementi di una collezione (non è nelle slide)	`hasNext()` e `next()`, `Iterable`, il ciclo `for (x : collezione)`
Command	Una richiesta trasformata in un oggetto (non è nelle slide)	un'interfaccia con `execute()`, oggetti comando messi in coda o in una lista, spesso con `undo()`

I pattern non risolvono tutti i problemi

Le slide chiudono con una provocazione: «i design pattern risolvono tutti i vostri problemi di ingegneria del software». Non è così: prima si definisce il **problema**, poi si cerca il pattern che lo risolve. Un pattern applicato dove non serve aggiunge solo classi e complessità; in un esercizio, il nome del pattern va sempre **motivato** con il problema del testo e con la struttura del codice.